



ISE 105-Programlamaya

Giriş
Hafta 1 - Giriş

Ders Hakkında

Programlamaya Giriş dersi genel içerik:

- Bir programlama probleminin çözümü için gerekli ilke ve evreleri kavrayabilme.
- Bir problemin çözümü için gerekli algoritma ve akış şemalarının oluşturulması.
- Programlama dili kullanılarak, bir programlama dilinin yapısını anlayabilme ve kullanabilme.
- Algoritma ve akış şemaları hazırlanan problemlerin bir programlama dili kullanarak kod yazımını yapabilme.
- Yapısal programlama (Değişkenler, karar yapıları, döngüler, diziler, alt programlar, yapı gibi kavramları anlayabilme ve kullanabilme.)
- Nesneye Dayalı Programlamaya giriş (sınıflar, nesneler ...)

Bu dersin temel çıktısı, Bu dersin amacı, bilgisayar programlama konusuna bir giriş yapmak, programlama ile ilgili genel kavramları ortaya koymak, algoritma kavramı, algoritmaların nasıl oluşturulacağı ve yapısal programlama konusuna değinmektir.

Beklenen Hedef

Programlama konusunda tecrübe kazanmak ve en az bir programlama dili hakkında iyi derecede beceri kazanmak.

İçerik Örnekleri

C++ dili kullanılarak çözümler üretmek.



Değerlendirm e Sistemi

•Yıllık Çalışmaları

- Kısa Sınav 1 - %20
- Ödev 1 - %30
- Ara Sınav (Vize) - %50

//Detayları paylaşılacak

•Final Değerlendirmesi

- Yıl içi - %50
- Final Sınavı - %50



Ders Akışı



Algoritma Kavramları

Algoritmalar
Akış Diyagramları



Programlama Kavramları

Değişkenler Karar yapıları
Döngüler, Diziler
Alt programlar, Yapılar vs.

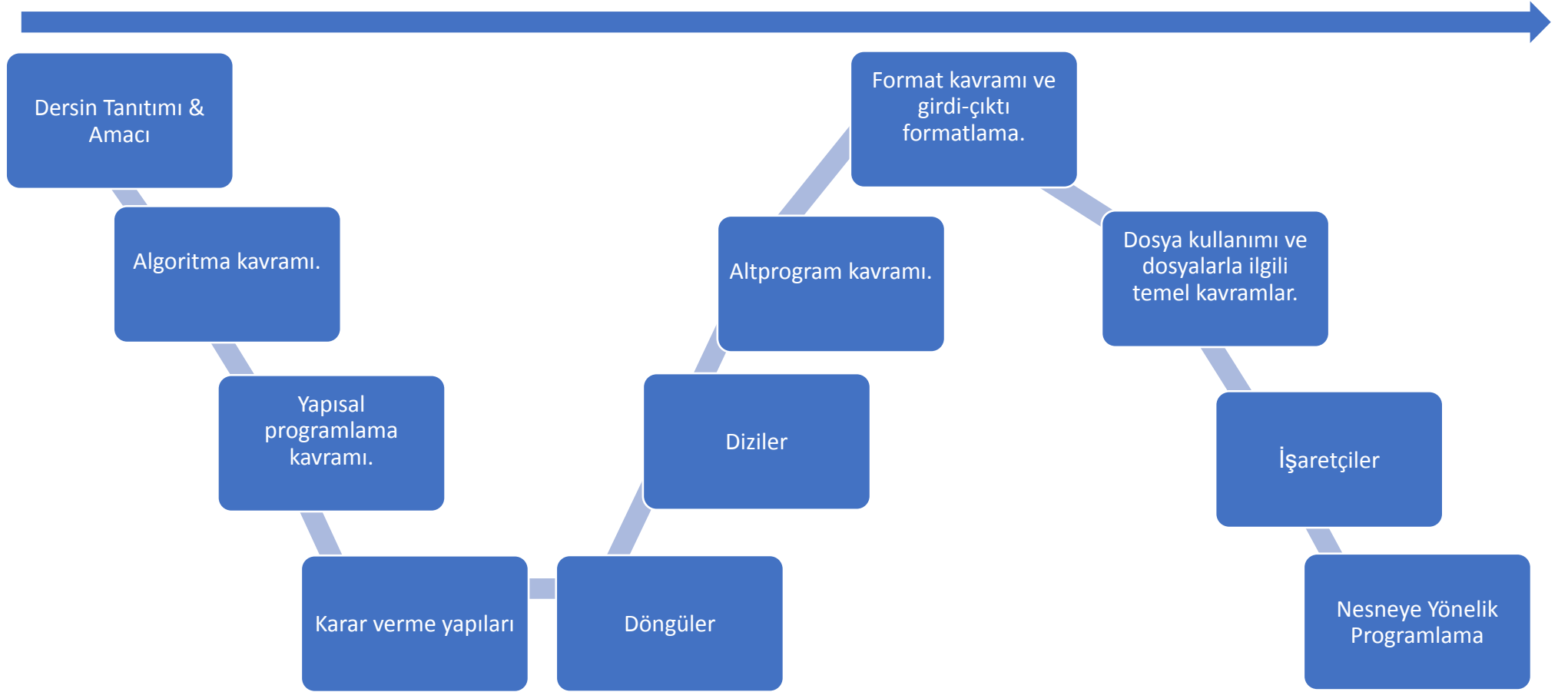


İşaretçiler ve Dosyalama

İşaretçiler
Dosyalama



Nesneye Yönelik Programlama



Geliştirme Ortamı



MS Visual Studio – (Ders boyunca geliştirilecek olan örnekler için Visual Studio geliştirme ortamı kullanılmaktadır.)

❑ Ücretsiz «Community» versiyonunu aşağıdaki linkten temin edebilirsiniz.

<https://visualstudio.microsoft.com/tr/free-developer-offers/>

❑ Öğrenci lisansınızı kullanarak da indirebilirsiniz. Gerekli adımlar için aşağıdaki link ziyaret edilebilir:

<http://bidb.sakarya.edu.tr/tr/duyuru/goster/82803/microsoft-azure-kullanimi-hakkinda>

• **Kullanılabilir Diğer Geliştirme Ortamları**

❑ Dev C++

<https://sourceforge.net/projects/orwelldevcpp/>

❑ Visual Studio Code

<https://visualstudio.microsoft.com/tr/free-developer-offers/>

❑ Code::Block

<https://www.codeblocks.org/>

❑ Net Beans IDE

<https://netbeans.apache.org/>

....

Tartışma Konuları

Tartışma Konuları

- Temel Kavramlar Programlama Kavramları
- Bilgisayarların Tarihçesi
- C++
- Algoritmalar ve Yazılım Geliştirme
- Programlama Dilleri
- Yazılımların Çalıştırılması
- Programlama (Yapısal, Nesneye Yönelik)
- Programlama Dilleri Tarihçe
- Yeni Nesil Diller



Temel Kavramlar

Bilgisayar

Genel olarak, girilen veriler üzerinde aritmetiksel, mantıksal ve ilişkisel işlemler yapabilen ve verileri depolama yeteneğine sahip olan elektromekanik cihazlara **bilgisayar** denir.

Donanım

Bilgisayarı oluşturan fiziksel parçalar

Mikroişlemci

Bellek

Giriş / Çıkış Birimi

Yazılım

Gerçek dünya problemlerini bilgisayar yardımıyla çözmek için **yazılım (program)** geliştirilir.

Probleme ilişkin çözümlerin, herhangi bir programlama dili kullanılarak, bilgisayarın anlayacağı komutlar dizisi şeklinde yazılmasına **program**, bilgisayarlara ne yapmaları gerektiğini bildiren ve iyi tanımlanmış işlevleri yerine getirmek üzere tasarlanmış programların, prosedürlerin, verilerin veya talimatların bir koleksiyonuna da **yazılım** denir.



Zamanda Yolculuk



Endüstri 1.0'dan Endüstri 4.0'a

Birinci Endüstri devrimi

Su ve buhar gücünün yardımıyla mekanik üretim tesislerinin ortaya çıkması



İlk mekanik dokuma tezgahı, 1784

İkinci Endüstri devrimi

Elektrik enerjisinin yardımıyla imalat iş bölümü ve seri üretimin ortaya çıkması



İlk montaj hatları, Cincinnati'deki mezbahalar, 1870

Üçüncü Endüstri devrimi

Üretimi daha da otomatikleştiren elektronik ve BT sistemlerinin kullanılmaya başlanması



İlk programlanabilir mantık devresi (PLC), Modicon 084, 1969

Dördüncü Endüstri devrimi

Siber-fiziksel sistemlerin kullanılmaya başlanması



Karmaşıklık derecesi

Zaman

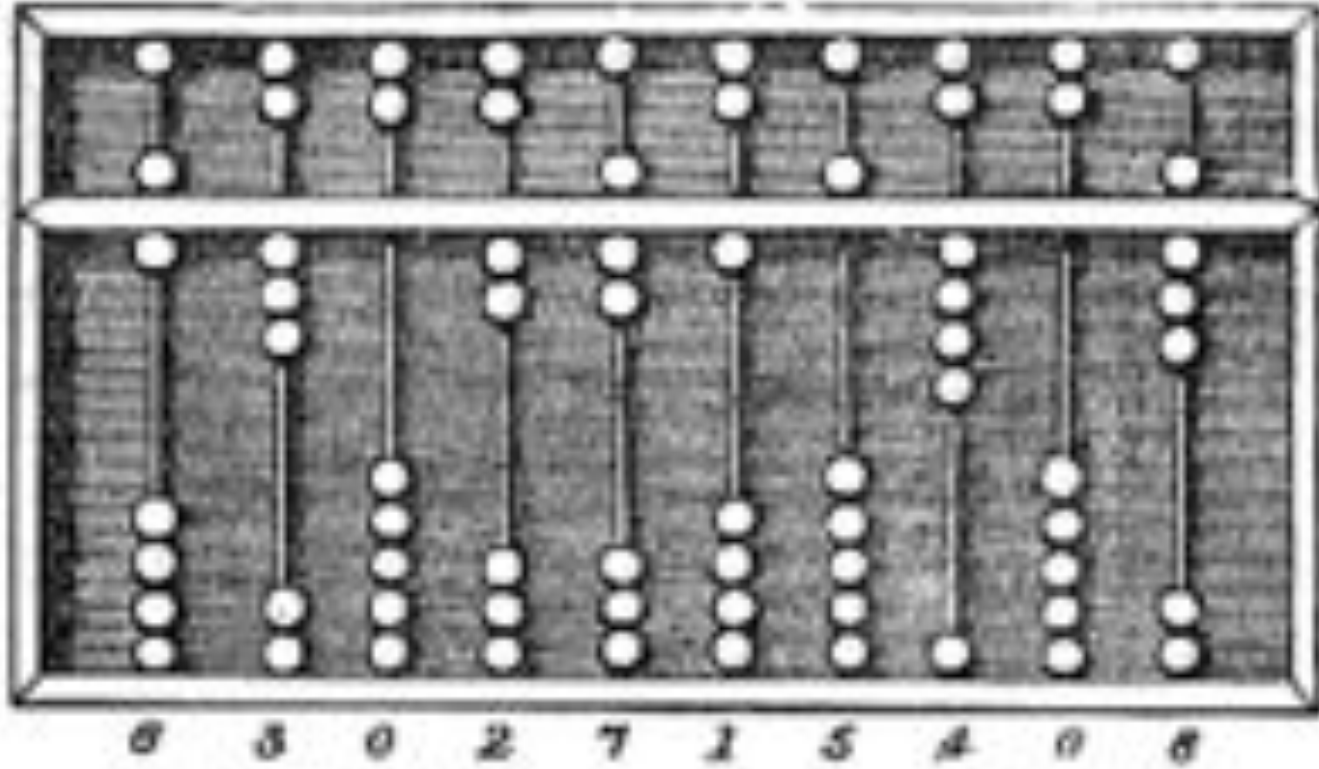
Dördüncü Sanayi Devrimi :

Yapay Zeka Uygulamaları, Karanlık Fabrikalar, Bulut Teknolojisi, Nesnelerin İnterneti

Zamanda Yolculuk



ABAKÜS



Zamanda Yolculuk

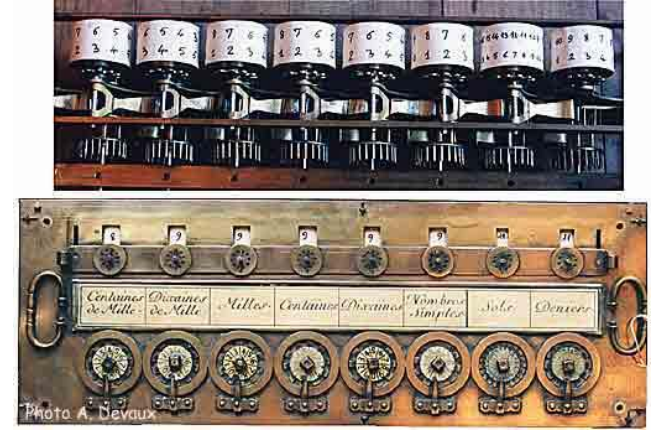


MEKANİK

- Blaise Pascal Toplama Makinesi (17. yy. ortaları)



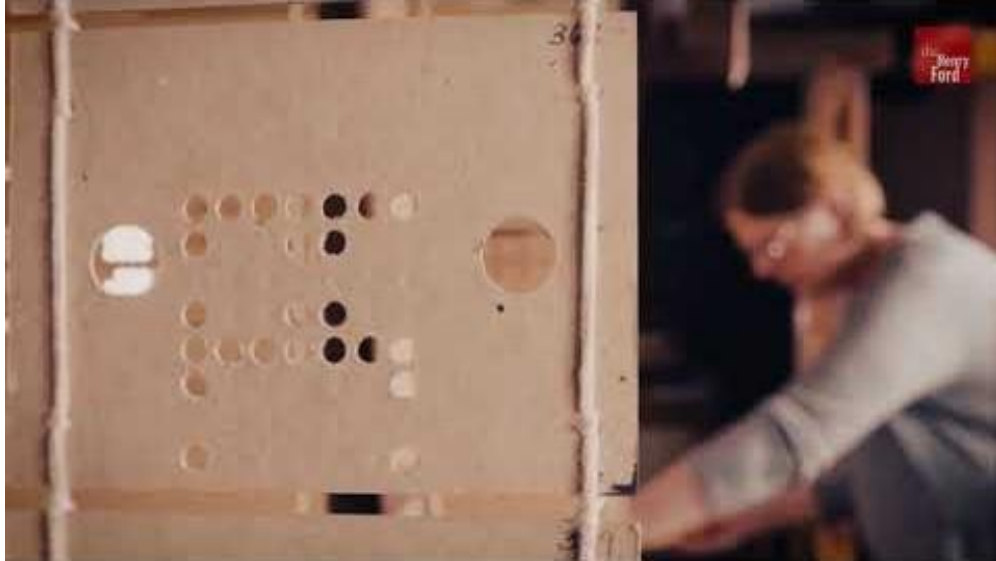
<https://youtu.be/SeyMTzKYKqg>



Zamanda Yolculuk

JACQUARD'IN DOKUMA TEZGAHI

Jacquard kendi dokuma tezgahlarında bir desenden birden fazla ve hızlı olarak dokumak için kartlar üzerine desen ve renkleri tanımlayarak dokuma tezgahlarını programlama yoluna girmiştir. Jacquard, dokuma tezgahını denetleyen ve delikli kartlar esasına göre çalışan bir mekanizma kurmuştur. Her karttaki delik kalıbı kullanılacak ipliğin rengini belirliyordu.



<https://youtu.be/MQzpLLhN0fY>

<https://youtu.be/K6NgMNvK52A>

Zamanda Yolculuk

MEKANİK BILGISAYARLAR

- Charles Babbage'ın geliştirdiği Analitik Motor adı verilen sistem üzerinde ilk çalışmaları yapan Ada Lovelace ilk programcı olarak kabul edilmektedir.
- 2009 yılından beri her Ekim ayının ikinci Salı günü "Ada Lovelace Day" (*Ada Lovelace Günü*)
- 1890'da Herman Hollerith delikli kartların kullanılarak verilerin işlendiği Hollerith Tabulatör(Listeleyici) tasarladı. ABD nüfus Bürosunun veri hesaplamaları böylece 10 yıldan 2.5 yıla düştü.



ADA LOVELACE
FIRST COMPUTER PROGRAMMER

The Analytical Engine

Lovelace's program turned a complex formula into simple calculations that could be encoded on punched cards and fed into Charles Babbage's Analytical Engine, a mechanical computer that he designed but never built. She published it in 1843, a century before the modern computer age.

"I want to put in something about Bernoulli's Number, in one of my Notes, as an example of how an explicit function may be worked out by the engine, without having been worked out by human head and hands first."

$$\frac{x}{e^x - 1} = \frac{1}{1 + \frac{x}{2} + \frac{x^2}{2 \cdot 3} + \frac{x^3}{2 \cdot 3 \cdot 4} + \&c.}$$

A Universal Computer

Lovelace did more than write the first computer program. She was also the first person to realise that a general purpose computer could do anything, given the right data and instructions.

"The Analytical Engine weaves algebraic patterns just as the Jacquard loom weaves flowers and leaves."

"Supposing, for instance, that the fundamental relations of pitched sounds in the science of harmony and of musical composition were susceptible of such expression and adaptations, the engine might compose elaborate and scientific pieces of music of any degree of complexity or extent."

Augusta Ada King, Countess of Lovelace
Born: 10 December 1815
Died: 27 November 1852

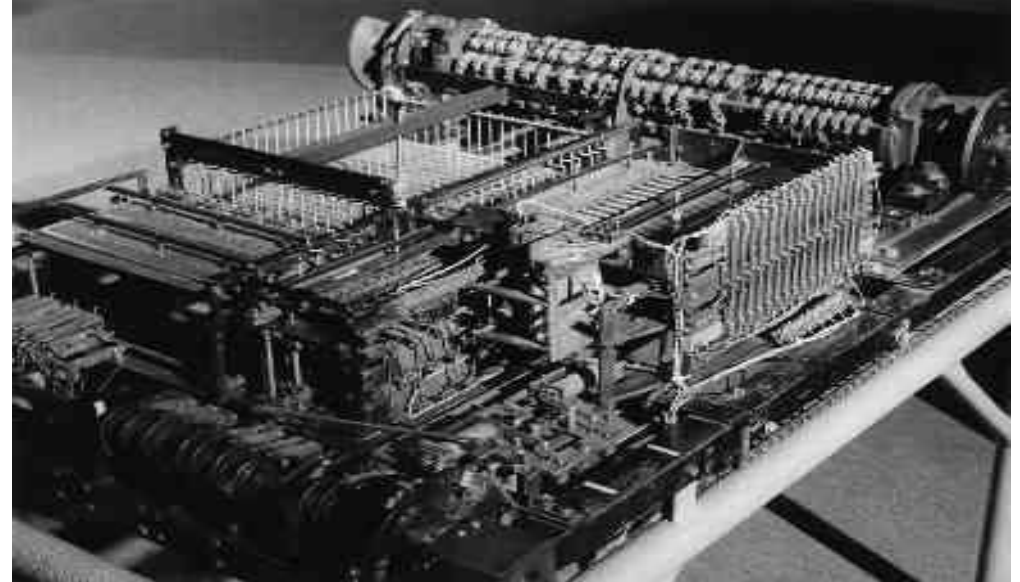
Ada Lovelace Day
FindingAda.com

Zamanda Yolculuk



ELEKTRİKLE ÇALIŞAN MEKANİK BİLGİSAYARLAR

1941'de Konrad Zuze Z3 isimli elektrik motorları ile çalıştırılan mekanik bir bilgisayar yaptı. Bu (Z1, Z2, Z3 ve Z4 serisi) program kontrollü ilk bilgisayardır.

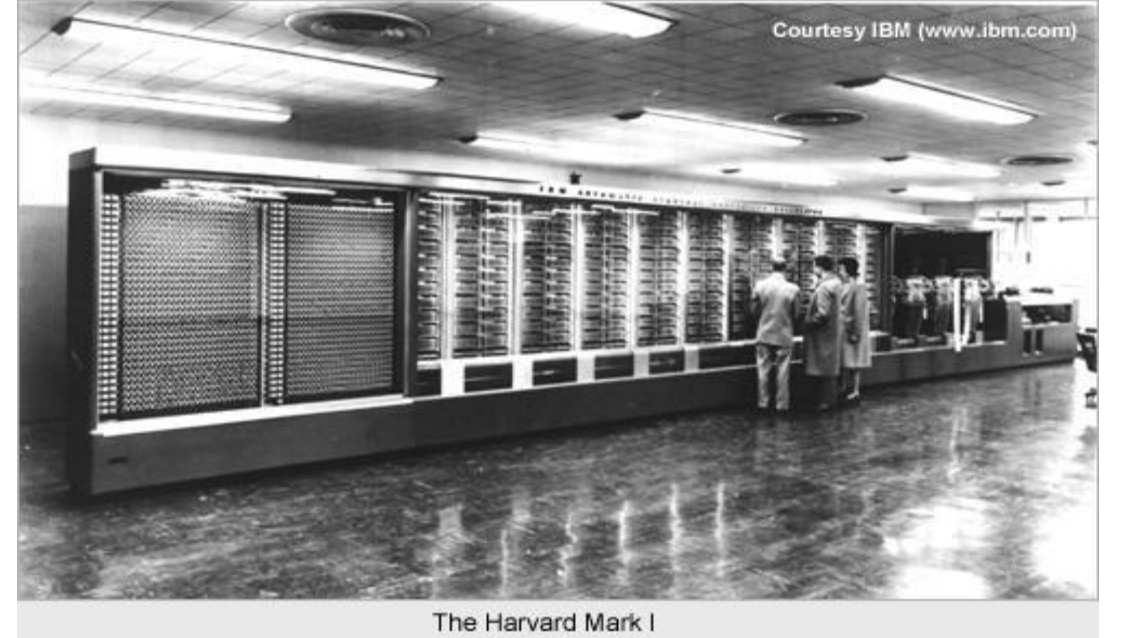


Zamanda Yolculuk



ELEKTRİKLE ÇALIŞAN MEKANİK BİLGİSAYARLAR

1944'de Harvard Üniversitesinde MARK I denilen bir bilgisayar geliştirildi. MARK 1, tamamı elektronik olmayan (Mekanik röleler vb.) genel amaçlı bir bilgisayardı. Bu makine 23 haneli iki sayıyı 4.5 saniyede çarpabiliyordu ve 2.4m yüksekliğinde olup üzerinde 800km uzunluğunda kablo kullanılmıştı.



The Harvard Mark I

Zamanda Yolculuk

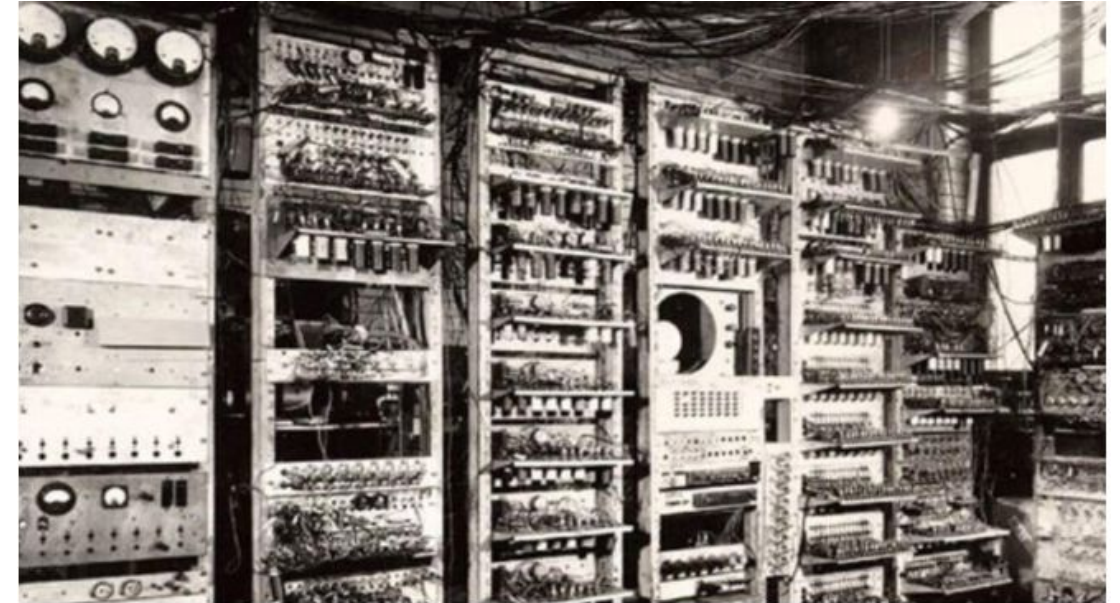


ELEKTRİKLE ÇALIŞAN MEKANİK BİLGİSAYARLAR

1946'da Pensilvanya Üniversitesinde ENIAC (Electronics Numerical Integrator and Calculator) geliştirildi.

Anahtar setlerinin, fişlerin ve soketlerin değiştirilmesi esasına göre çalıştığından ilk genel amaçlı bilgisayardır. 70 bin direnç, 10 bin kondansatör, 18000 lamba ve bu elemanların harcadığı 150-200 kilowatt enerji ve sadece 20 sayısı depolama özelliğine sahipti ve 30 tondur.

ABD'nin II. Dünya Savaşı'na katılmasıyla birlikte ordu tarafından gizli olarak Pennsylvania Üniversitesi'ne ait elektrik mühendisliği okulu Moore School of Electrical Engineering'e verildi. Amaç daha az isabet hatalı uzun menzilli top ve füzelerin hesaplamalarında kullanılmasıydı. O günlerde 60 saniye yol alan bir topun rotasının hesaplanması için, yaklaşık 20 saatlik çalışma gerekiyordu. ENIAC bu hesaplama süresini 15 saniyeye indirdi.



Zamanda Yolculuk



ELEKTRONİK BİLGİSAYARLAR

1946'de Dr. Von Neumann ve arkadaşları, programı bellekte saklayabilen ilk bilgisayar olan EDVAC (Elektronik Discrete Variable Automatic Computer) geliştirdi. 4096 bellek gözü bulunmaktaydı ve veriler ile programlar aynı bellekte saklanmaktaydı.

1948'de ilk transistor Bell lâboratuarlarında geliştirildi.



1951'de UNIVAC 1 adlı ilk ticari amaçlı olan bilgisayar geliştirildi. Bütün komutlar ve veriler 0 ve 1 şeklinde depolandı.

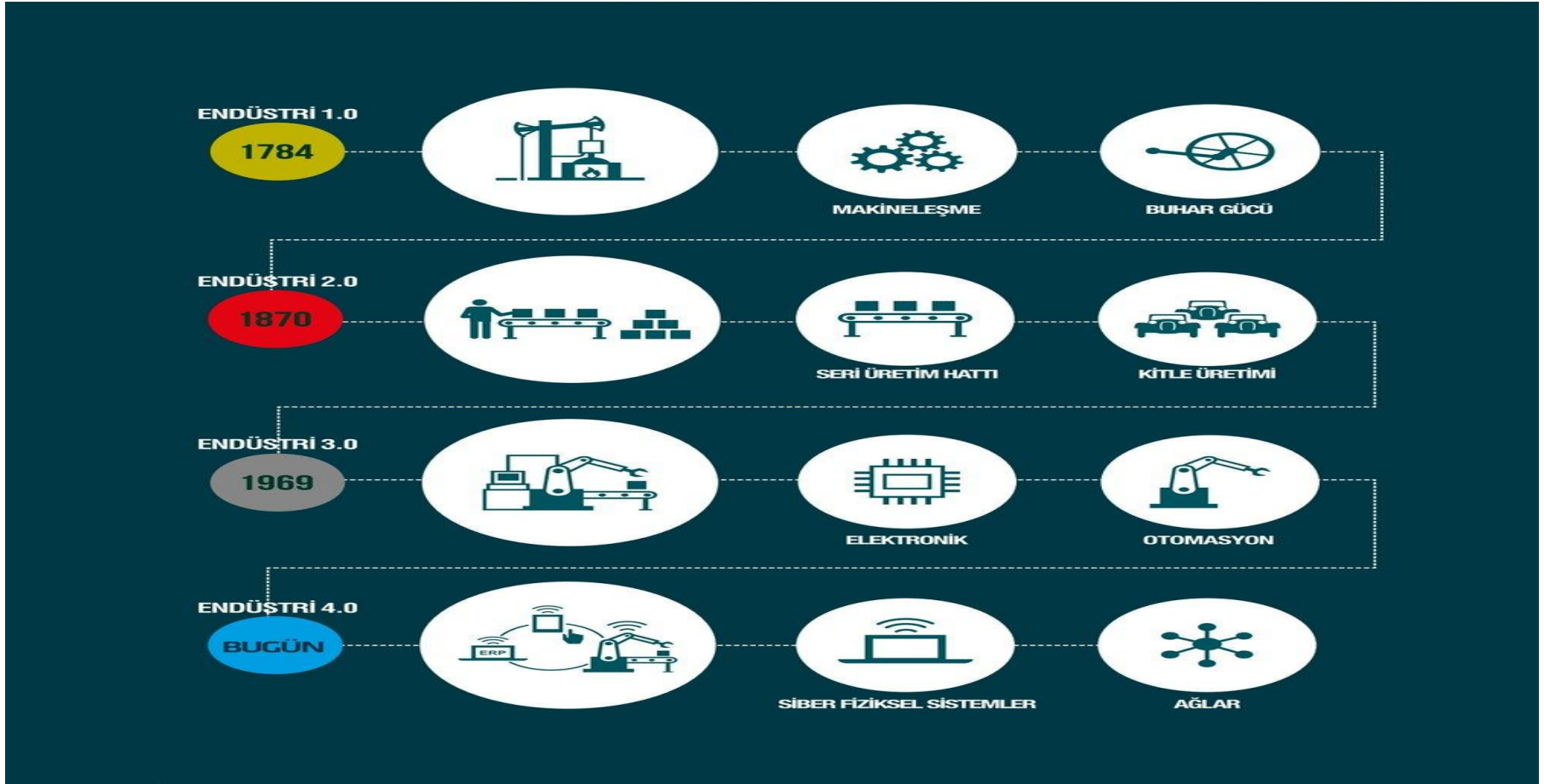


Zamanda Yolculuk



ELEKTRONİK BILGISAYARLAR

1958 de entegre devreler geliştirildi.



Endüstri 4.0



•Yapay Zekâ Uygulamaları

Bilgisayarların çeşitli faaliyetleri zeki canlılara benzer şekilde yerine getirme kabiliyeti

•**Karanlık Fabrikalar**

otomatik sistemlerle donatılmış ve bünyesinde insanın varlığına ihtiyaç duymayan yapılar

•**Bulut Teknolojisi**

sunucu, depolama, veri tabanı, ağ, yazılım, analiz ve makine zekâsı gibi bilgi işlem hizmetlerinin, internet tabanlı bulut bilişim ağı ile kullanıcılar arasında kaynak paylaşımını sağlayan bir online hizmet türüdür.

•**Nesnelerin İnterneti**

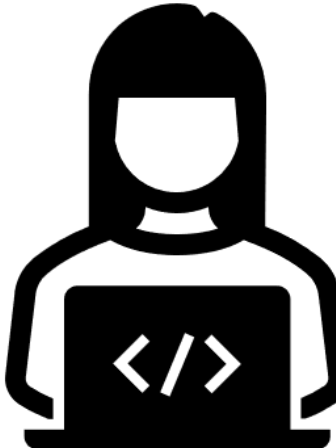
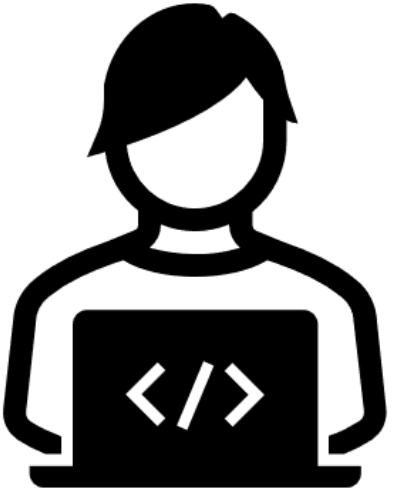
benzersiz bir şekilde adreslenebilir nesnelerin kendi aralarında oluşturduğu, dünya çapında yaygın bir ağ ve bu ağdaki nesnelerin belirli bir protokol ile birbirleriyle iletişim içinde olmaları

TEMEL KAVRAMLAR

Programlama

Günlük hayatta karşılaşılan genel veya özel bir problemin makineler ile çözülmesi istendiğinde öncelikle problemin gerçek hayattan soyutlanıp makineye anlatılması gerekir.

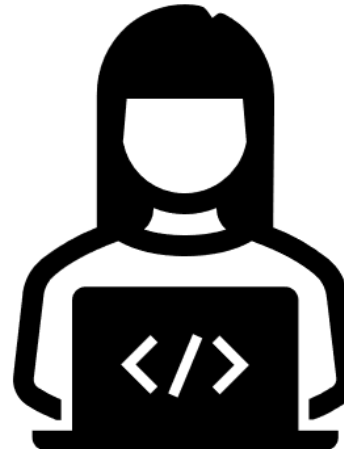
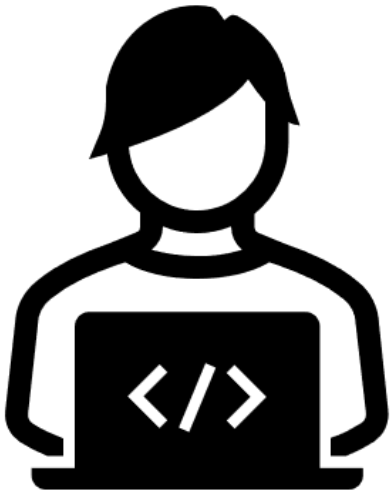
Bu problemlerin makinelere tanıtılması, öğretilmesi ve öğretilen çözüm yolları ile sonuca ulaştırılmasını sağlama süreçlerinin tamamını kapsayan süreç **programlama** olarak adlandırılır.



TEMEL KAVRAMLAR

Programlama Dili

- Günlük hayatta karşılaşılan genel veya özel bir problemin makineler ile çözülmesi istendiğinde problemin gerçek hayattan soyutlanıp makineye anlatılmasını sağlayan araçlardır.
- Programlama dilleri, yazılımcının bilgisayara neyi nasıl yapacağını hangi koşullarda hangi işlemlerin yapılacağını tam olarak anlatmasını sağlar.
- Her programlama dili diğerinden farklıdır. Her birinin farklı komutları ve kuralları mevcuttur. En iyi programlama dili budur diye bir şey söyleyemeyiz. Çünkü her programlama dilinin bir diğerine göre üstün ve zayıf tarafları vardır.
- Günümüzde en yaygın kullanılan programlama dilleri Java ,C ,C++ ,C#, PHP, Python, JavaScript,



C++



- 1979 Bjarne Stroustrup tarafından geliştirilen genel amaçlı bir dil.
- Nesne yönelimli ve Yapısal Programlama gibi programlama yöntemlerine imkan vermekte.
- İlk olarak «C with Classes» olarak adlandırılmış 1983 yılında «C++» olarak değiştirilmiştir.
- C dilinden türetilmiştir. C dilinin üst kümesi olarak düşünülebilir.
- Sınıflar, nesneler ve nesneye yönelik programlama C++ üzerinde C'ye eklenen en önemli özellikleri oluştururlar.
- Bununla birlikte yeni birçok özellik eklenmiştir.(Geliştirilmiş I/O yöntemleri vb.)
- Dolayısıyla Nesne yönelimli programlamada yer alan kapsülleme, çok biçimlilik, kalıtım gibi imkanlara sahiptir.

(Bu özelliklere ilerleyen haftalarda değineceğiz)

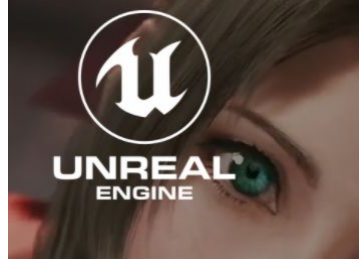
- **Programlama dilleri donanıma müdahalesine göre** düşük, orta ve yüksek seviyeli olarak sınıflandırıldığında C++ bellek yönetimi yapmaya imkan verdiği için dolayı orta seviyeli bir programlama dili denilebilir.

C++ Örnek Kullanım Alanları



Oyunlar

C ++ donanıma yakındır, kaynakları kolayca değiştirebilir, CPU yoğun işlevler üzerinden prosedürel programlama sağlar ve hızlıdır.



GUI (graphical User Interface) tabanlı Uygulamalar

Veritabanı Yazılımları

C ++ veritabanı yönetim yazılımı yazılırken de kullanılır. –MySQL veritabanı.



İşletim Sistemleri

C ++, düşük düzeyli programların yazılmasına da yardımcı olan geniş bir sistem düzeyinde işlev koleksiyonuna sahiptir.

Birçok Microsoft yazılımları

Win XP, Microsoft Office, Internet Explorer....

Microsoft Windows OS



Mozilla Firefox



Tarayıcılar

C++ Örnek Kullanım Alanları



İleri Hesaplama ve Grafik

C++, yüksek performanslı görüntü işleme, gerçek zamanlı fiziksel simülasyonlar ve yüksek performans ve hız gerektiren mobil sensör uygulamaları gerektiren bir uygulama geliştirmede kullanışlıdır.



Bulut/Dağıtık Sistemler

Donanıma olan yakınlıkları sebebiyle C++ tercih edilebiliyor.

Bloomberg



Derleyiciler (Çeşitli yüksek-seviye programlama dillerinin derleyicileri)

Gömülü Sistemler (Akıllı saatler, tıbbi ekipman sistemleri..)

Kurumsal Yazılımlar

Alias System



Kütüphaneler (birçok yüksek-seviye dilin kütüphaneleri arka planda C++ temellidir.)

TIOBE INDEX

<https://www.tiobe.com/tiobe-index/>

Sep 2025	Sep 2024	Change	Programming Language		Ratings	Change
1	1			Python	25.98%	+5.81%
2	2			C++	8.80%	-1.94%
3	4	▲		C	8.65%	-0.24%
4	3	▼		Java	8.35%	-1.09%
5	5			C#	6.38%	+0.30%
6	6			JavaScript	3.22%	-0.70%
7	7			Visual Basic	2.84%	+0.14%
8	8			Go	2.32%	-0.03%
9	11	▲		Delphi/Object Pascal	2.26%	+0.49%
10	27	▲		Perl	2.03%	+1.33%
11	9	▼		SQL	1.86%	-0.08%
12	10	▼		Fortran	1.49%	-0.29%
13	15	▲		R	1.43%	+0.23%
14	26	▲		Ada	1.27%	+0.56%
15	13	▼		PHP	1.25%	-0.20%
16	17	▲		Scratch	1.18%	+0.07%
17	21	▲		Assembly language	1.04%	+0.05%
18	14	▼		Rust	1.01%	-0.31%
19	12	▼		MATLAB	0.98%	-0.49%
20	18	▼		Kotlin	0.95%	-0.14%

PYPL

Popularity of Programming Language Index

<https://pypl.github.io/PYPL.html>

Worldwide, Sept 2025 :

Rank	Change	Language	Share	1-year trend
1		Python	29.69 %	+0.2 %
2		Java	14.72 %	-0.7 %
3	▲	C/C++	9.27 %	+2.5 %
4	▼	JavaScript	6.79 %	-1.4 %
5	▲	R	5.26 %	+0.6 %
6	▼	C#	4.81 %	-1.8 %
7	▲▲▲▲	Objective-C	4.17 %	+1.7 %
8	▼	PHP	3.34 %	-0.8 %
9	▲	Rust	2.73 %	+0.2 %
10	▼	Swift	2.72 %	+0.0 %
11	▼▼▼	TypeScript	2.45 %	-0.5 %
12	▲▲▲	Ada	2.11 %	+1.1 %
13	▼	Go	1.56 %	-0.6 %
14		Matlab	1.49 %	+0.0 %
15	▼▼	Kotlin	1.37 %	-0.6 %
16	▲	Powershell	1.1 %	+0.1 %
17	▲▲	Ruby	1.05 %	+0.1 %
18	▲▲	Lua	0.97 %	+0.2 %
19	▼▼▼	VBA	0.95 %	-0.1 %
20	▼▼	Dart	0.88 %	-0.1 %

TEMEL KAVRAMLAR

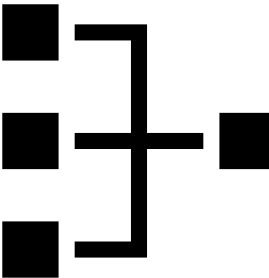
Algoritma

Bir problemin çözümüne yönelik işlem basamaklarının sıralı biçimde ve sonlu olarak ifade edilmesi

Bilgisayar dilinde “ bir sorunun çözümü için öngörülen işlemlerin mantıksal ve sembolik anlatımı”

9. yüzyılda yaşamış Türk-İslam matematikçi ve astronomu Harzemli Mehmet, toplama, çıkarma, ikiye bölme, bir sayının iki katını bulma, denklem çözümü, vb. gibi cebirsel işlemleri açıklayan bir çalışma yaptı. Batılılar, onun bu çalışmalarına Latince "algorismus" yani bugünkü adı ile **algoritma** dediler ve algoritma kavramı ilk defa burada kullanılmış oldu

Bir algoritma yazmak için mutlaka bir dile bağımlı kalma zorunluluğu yoktur. Önemli olan yazılan algoritmanın herhangi bir programlama diline uyarlanabilir olmasıdır.



TEMEL KAVRAMLAR

Akış Diyagramı

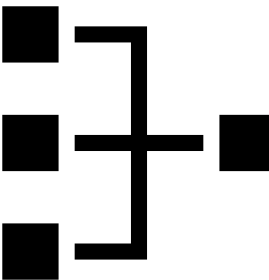
Algoritmaların özel geometrik şekiller ile gösterilmesidir

Sözde Kod

Algoritmaların ifade edilmesinde akış diyagramlarının yanı sıra, konuşma dili ile programlama dili arasında, **sözde kod** (pseudo-code) adı verilen bir araç kullanılır.

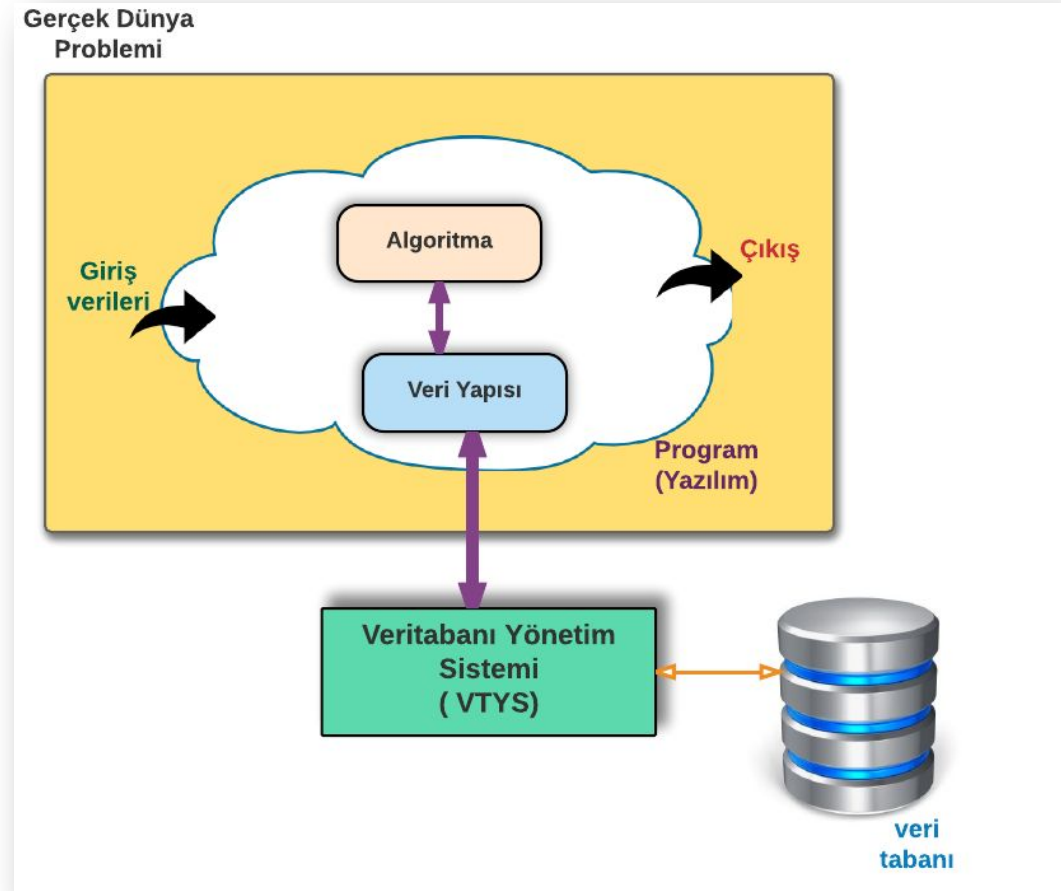
Programlama dilinden bağımsız olarak yazılan kodlardır, yani programlar gibi derlenmez veya işlenmezler.

Her programcı kendi sözde kodunu geliştirebilir . Fakat kişisel sözde kodlar başkaları tarafından anlaşılabilir bir biçimde açık olmalıdır.

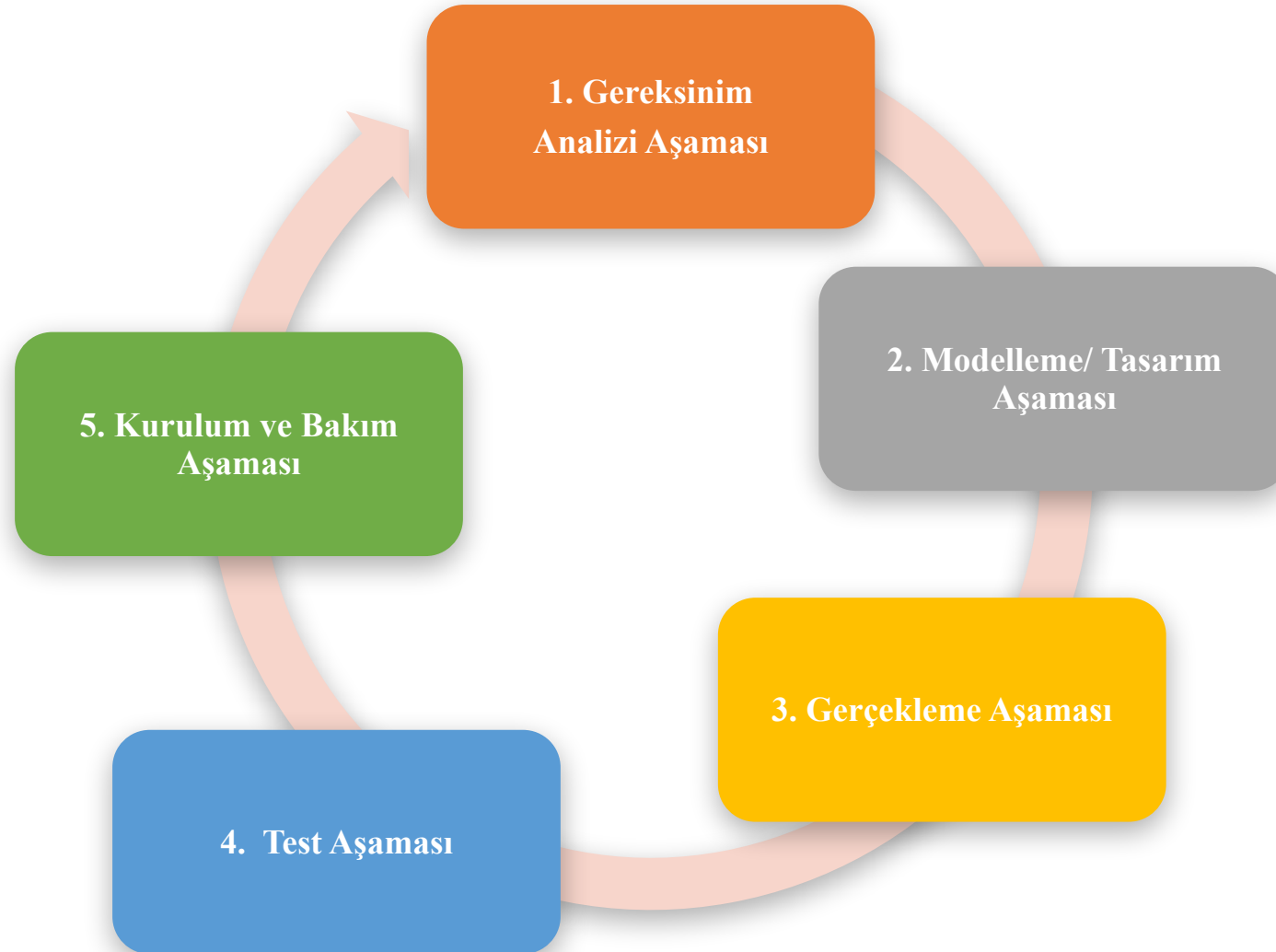


Büyük Resim – Algoritmik Problem

Gerçek dünya problemlerini bilgisayar yardımıyla çözmek için **yazılım (program)** geliştirilir.



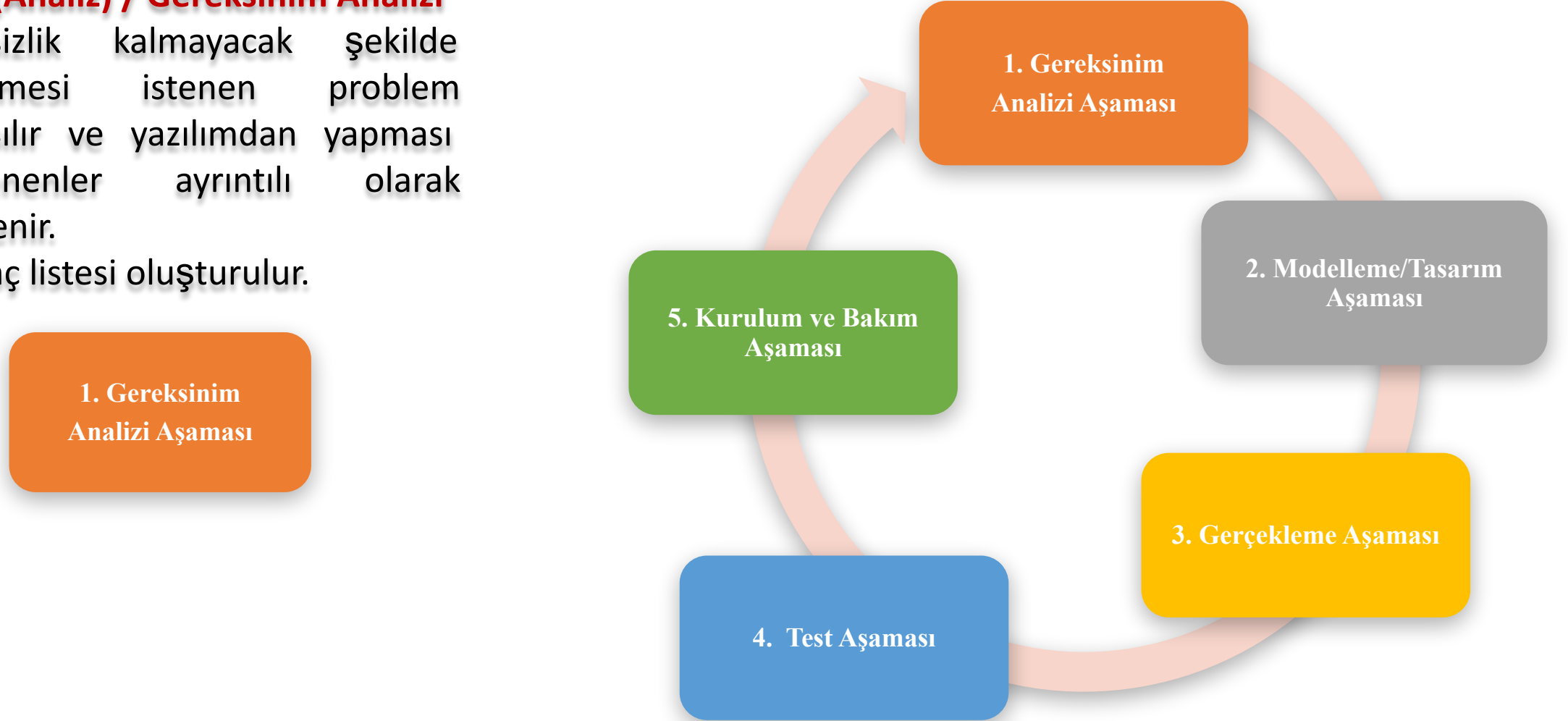
Yazılım Geliştirme Yaşam Döngüsü



Yazılım Geliştirme Yaşam Döngüsü

Çözümleme (Analiz) / Gereksinim Analizi

- Belirsizlik kalmayacak şekilde çözülmesi istenen problem anlaşılır ve yazılımdan yapması beklenenler ayrıntılı olarak listelenir.
- İhtiyaç listesi oluşturulur.



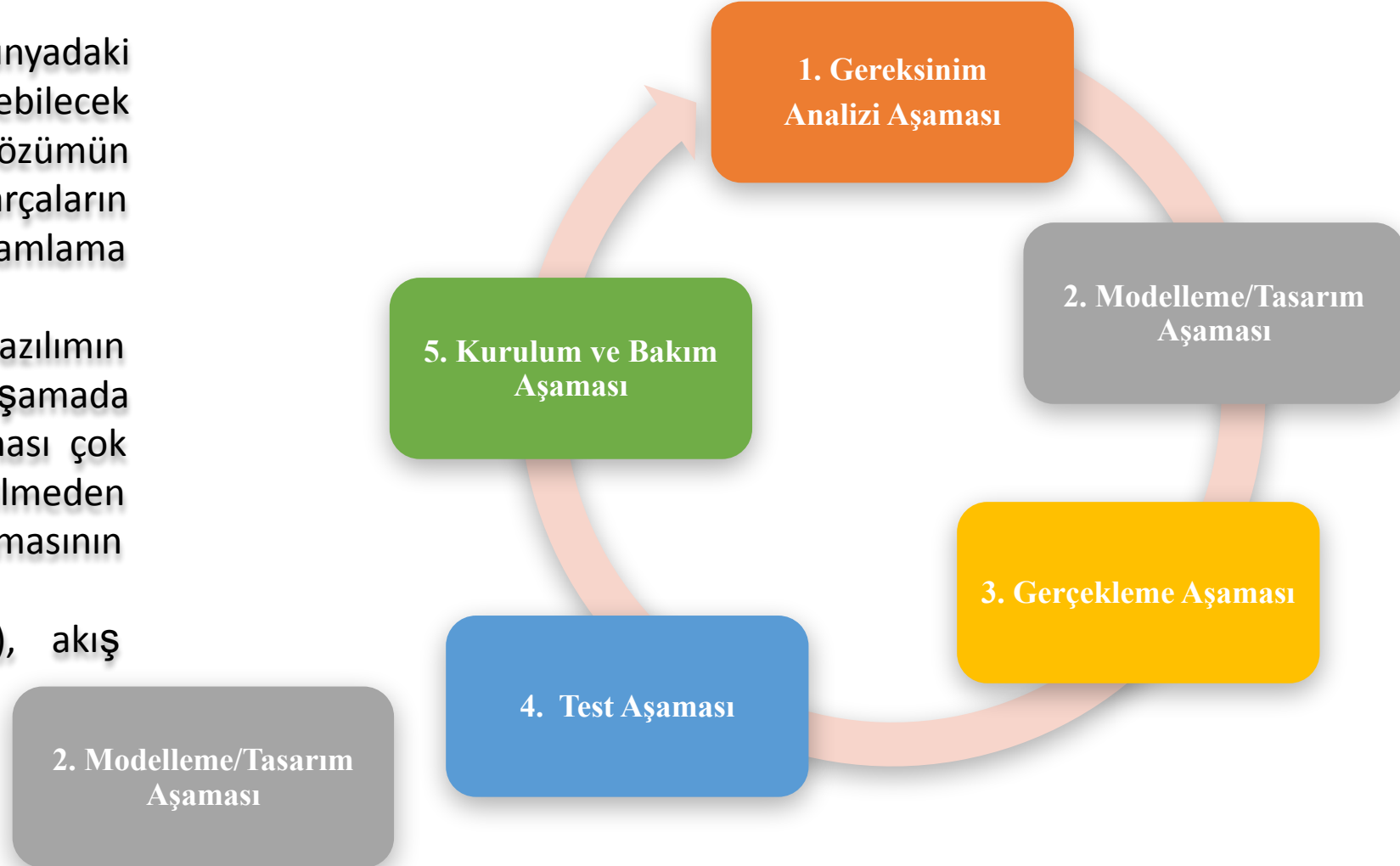
Yazılım Geliştirme Yaşam Döngüsü

Modelleme / Tasarım Aşaması

Bu aşamada amaç, gerçek dünyadaki problemin bilgisayarda temsil edilebilecek soyut bir modelinin oluşturulmasıdır. Çözümün hangi unsurlardan oluşacağı ve bu parçaların nasıl modelleneceği tamamıyla programlama yöntemine bağlıdır.

Tasarım sonrası ortaya çıkacak olan yazılımın kalitesini doğrudan etkilediğinden bu aşamada iyi ve doğru bir çözümün oluşturulması çok önemlidir. Bu nedenle kodlamaya geçilmeden önce kurulan modelin sağlamlasının (verification) yapılması gerekir.

UML (Unified Modelling Language), akış diyagramı, sözde kod, vs.

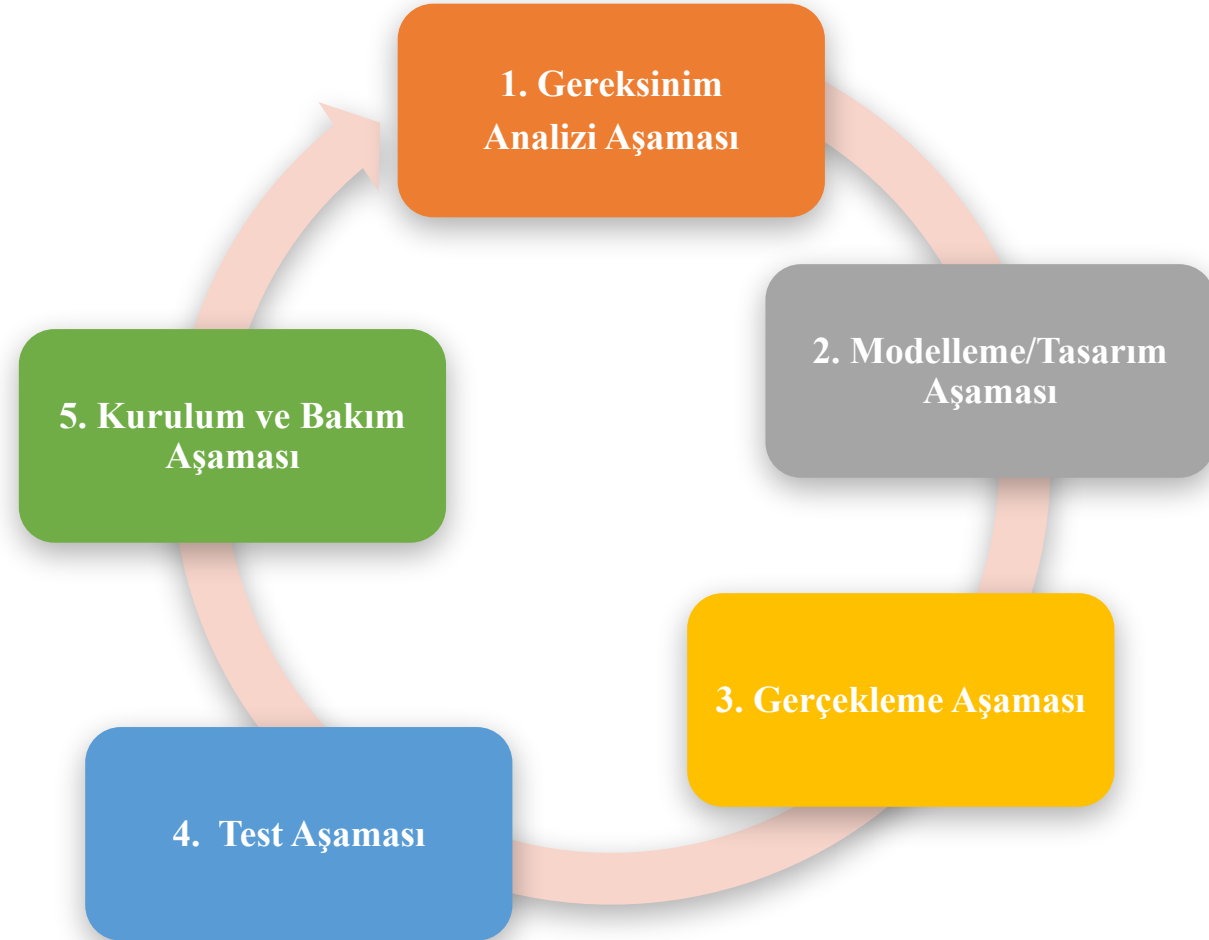


Yazılım Geliştirme Yaşam Döngüsü

Gerçekleme Aşaması

Tasarım aşamasında oluşturulan model, bir programlama dili ile bilgisayara bu aşamada aktarılır.

3. Gerçekleme Aşaması



Yazılım Geliştirme Yaşam Döngüsü

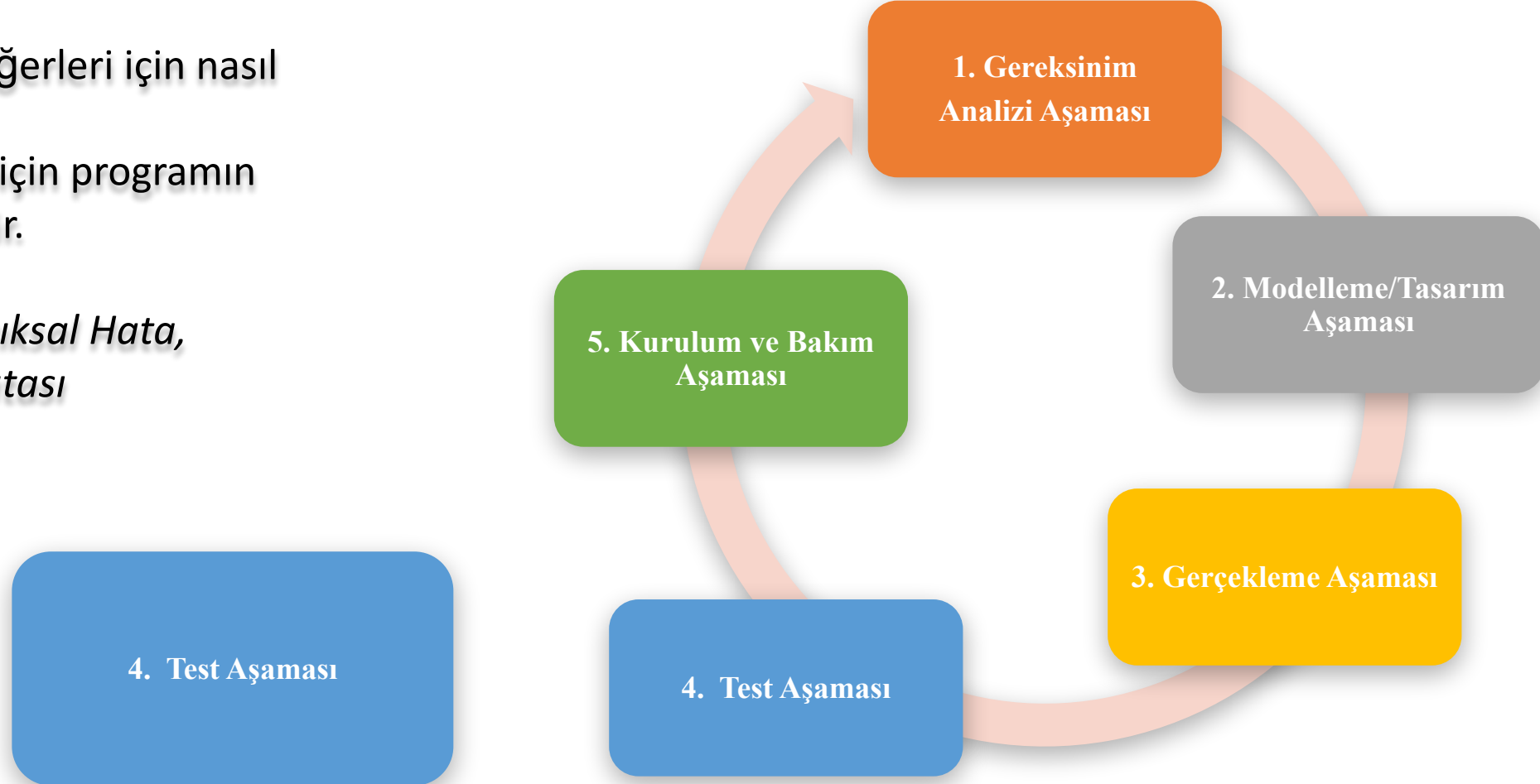
Test Aşaması

Programın olası giriş değerleri için nasıl davrandığı incelenir.

Özellikle kritik değerler için programın çalışması test edilmelidir.

Hata Ayıklama

*Yazım Hatası, Mantıksal Hata,
Çalışma Zamanı Hatası*

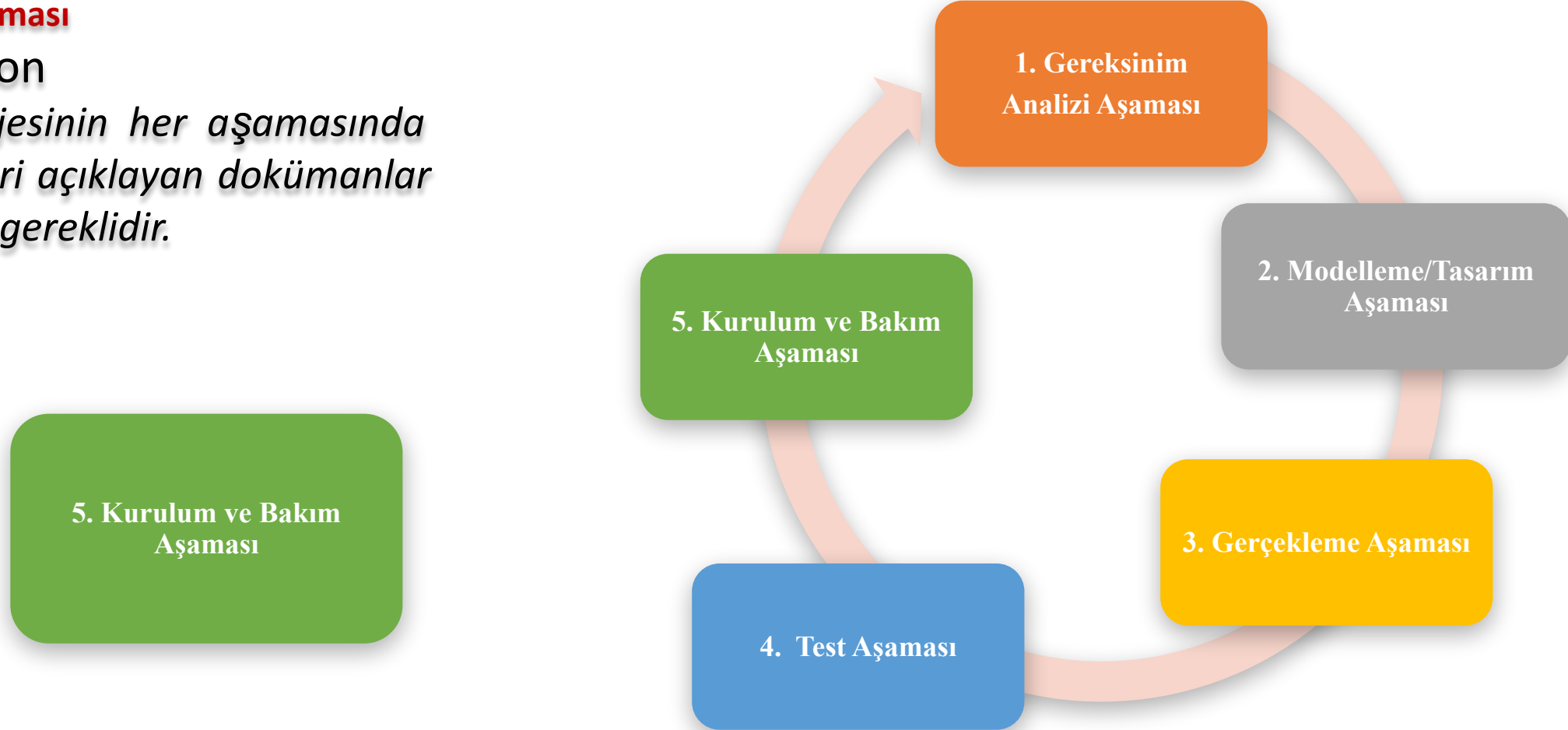


Yazılım Geliştirme Yaşam Döngüsü

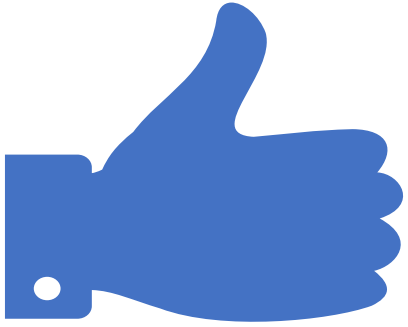
Kurulum ve Bakım Aşaması

Dokümantasyon

Yazılım projesinin her aşamasında yapılan işleri açıklayan dokümanlar hazırlamak gereklidir.



Yazılım Kalitesi



- Etkinlik
 - Hız ve kaynak kullanımı
- Bakım kolaylığı
- Sağlamlık
 - Hatalardan etkilenme
- Anlaşılabilirlik
- Taşınabilirlik
- Geliştirilebilirlik
- Güvenlik

Programlama Dilleri

Makine Dili (Düşük Seviye)

Geliştirilen ilk programlama dilidir

Anlaşılması çok zordur

Komutlar, ikili (0 ve 1) sayılardan oluşur.

Biraz daha kolay okunabilmesi için komutlar 16'lık sayı sistemi (hex) ile yazılır fakat derlenme sonrası kod ikilik sisteme çevrilir.

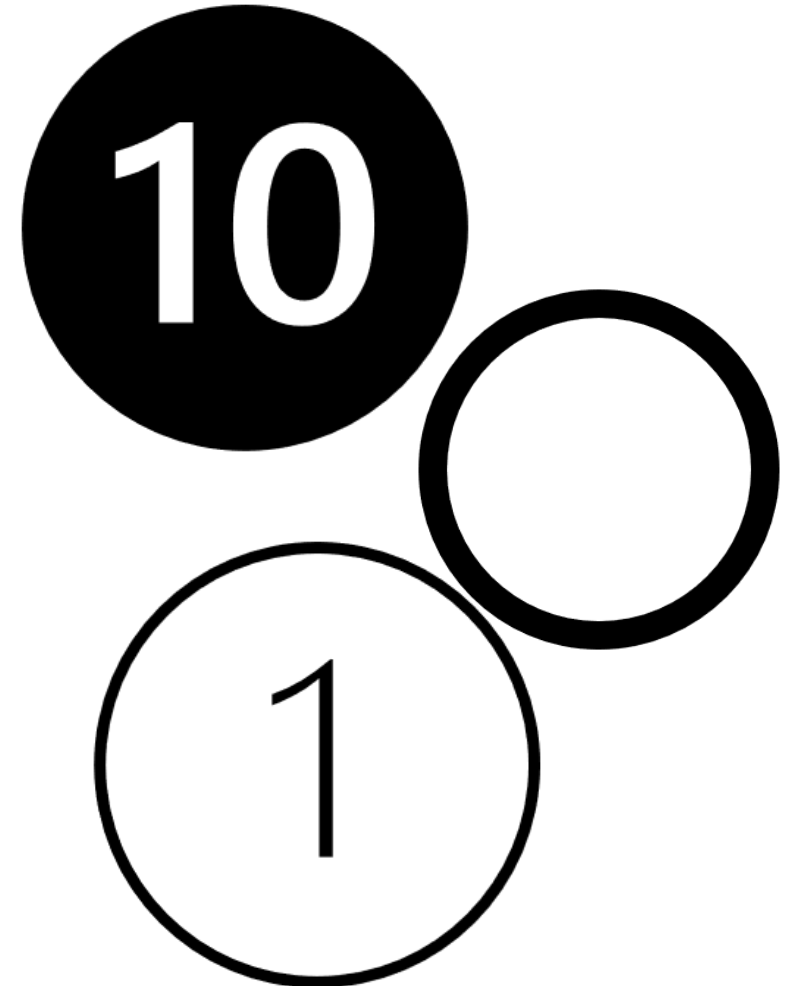
İşlemci ikilik tabandaki kodu okur ve uygular.

Makine (yani, donanım) bağımlıdır.

+1300042774

+1400593419

+1200274027



Programlama Dilleri

Assembly Dili (Orta Seviye)

İkinci kuşak dillerdir

Anlaşılması nispeten daha kolaydır

İngilizce kısaltmalardan oluşan semboller kullanılır

Makine bağımlıdır

Assembler derleyicisi ile makine koduna
dönüştürülür

LOAD sayı1

ADD sayı2

STORE toplam

LOAD

ADD

STORE

Programlama Dilleri

Yüksek Seviyeli Diller

- Anlaşılması çok daha kolaydır
- Yazılan programlar farklı makinelerde de kullanılabilirler.
- Assembly dili veya makine dilindeki birçok satır tek bir komutla gösterilir. Böylece program daha kısa bir sürede yazılır.
- Komutlar İngilizce kelimeler ve matematiksel ifadelerden oluşur
- Pascal, Basic, Fortran, C/C++, Java, vs.
- Derleyici tarafından makine koduna dönüştürülür

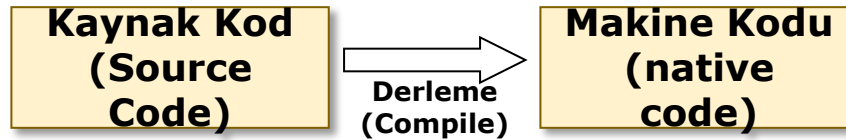
Yazılımların Çalıştırılması...

Derleme

Derlenen Dil / Compiled Language

Bir programlama dilinde yazılan kodu başka bir kod haline dönüştüren programlar

Pascal, C, C++, ...



Yorumlama

Yorumlanan Dil / Interpreted Language

Komutlar teker teker çalıştırılır

Komut okuma ve çevirme işlemi çalışma zamanında yapılır (düşük hız)

Hata düzeltme daha kolaydır

Derleyiciden kaynaklanan sınırlamalar kalktığı için daha esnek bir çalışma ortamı sunar

Perl, TCL, Basic, MATLAB, ...

Basitçe, bir kaynak kodu hedef koda çevirdikten sonra çalıştıran ve dolayısıyla koddaki hataları yakalama işlemini ve kodun iyileştirilmesini daha kod çalıştırmadan yapan çeviricilere **derleyici**, kodu satır satır veya bloklar halinde çalıştırıp sırası gelmeyen satırları hiç çalıştırmayan ve bu satırlardaki hataları hiçbir zaman göremeyen ve kodun bütününe ait iyileştirmeleri yapamayan çeviricilere de **yorumlayıcı** denir.

Yazılımların Çalıştırılması...

Karma (Hybrid)

Genelde bir ortam yazılan dilin çalıştırılmasına kadar geçen sürede ya bir derleyici yada bir yorumlayıcı kullanılmaktadır. Gelişmekte olan teknolojiyle iki programı birden kullanan diller de türemiştir.

Kodlar derlenerek byte koda dönüştürülür

Byte kod yorumlanarak çalıştırılır

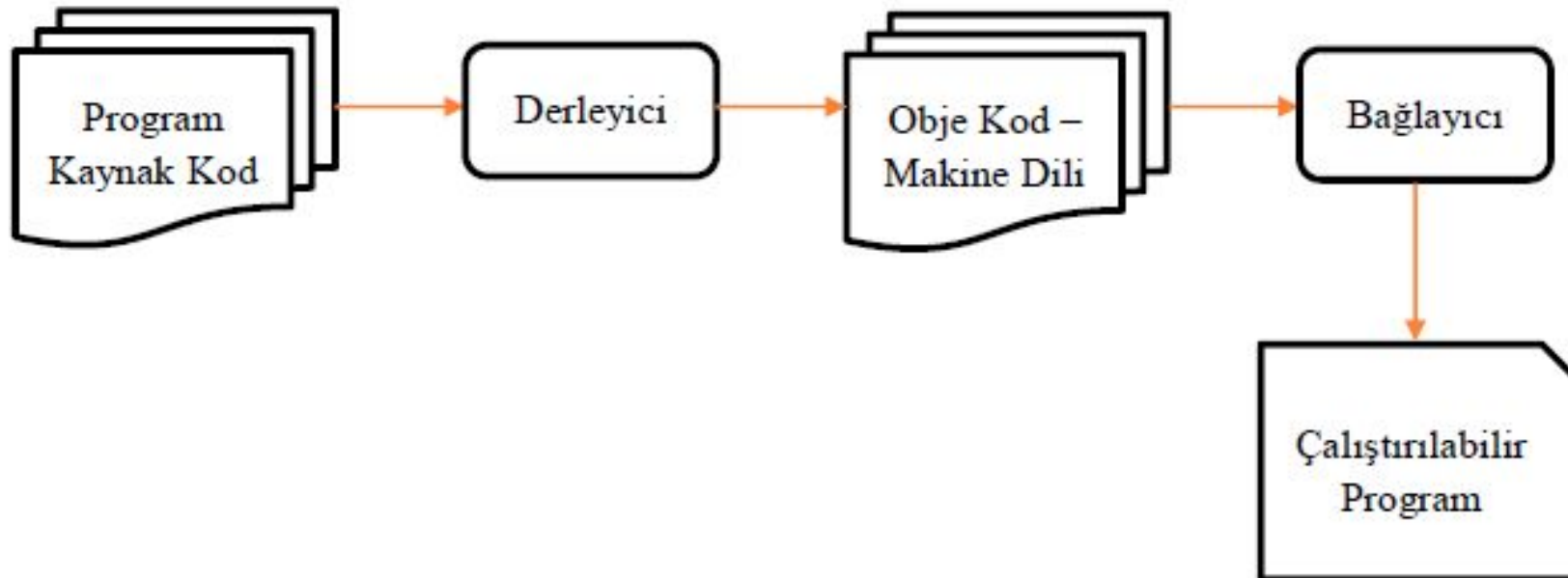
Java, Python

Java'da yazılan kod önce derlenerek byte code adı verilen ve sadece java sanal makinalarında (java virtual machine) çalıştırılabilen bir kod üretilmektedir. Bu üretilen ara kod daha sonra java sanal makinasında bir yorumlayıcı yapısına uygun olarak çalıştırılmaktadır.

Yazılımların Çalıştırılması...

Bağlayıcı (LINKER) Derleyici tarafından object dosyasına çevrilen bir veya birden çok dosyanın birbiri ile ilişkilendirilmesi ve tek bir çalıştırılabilir dosyaya (örneğin .exe) çevrilmesini sağlayan yazılımdır.

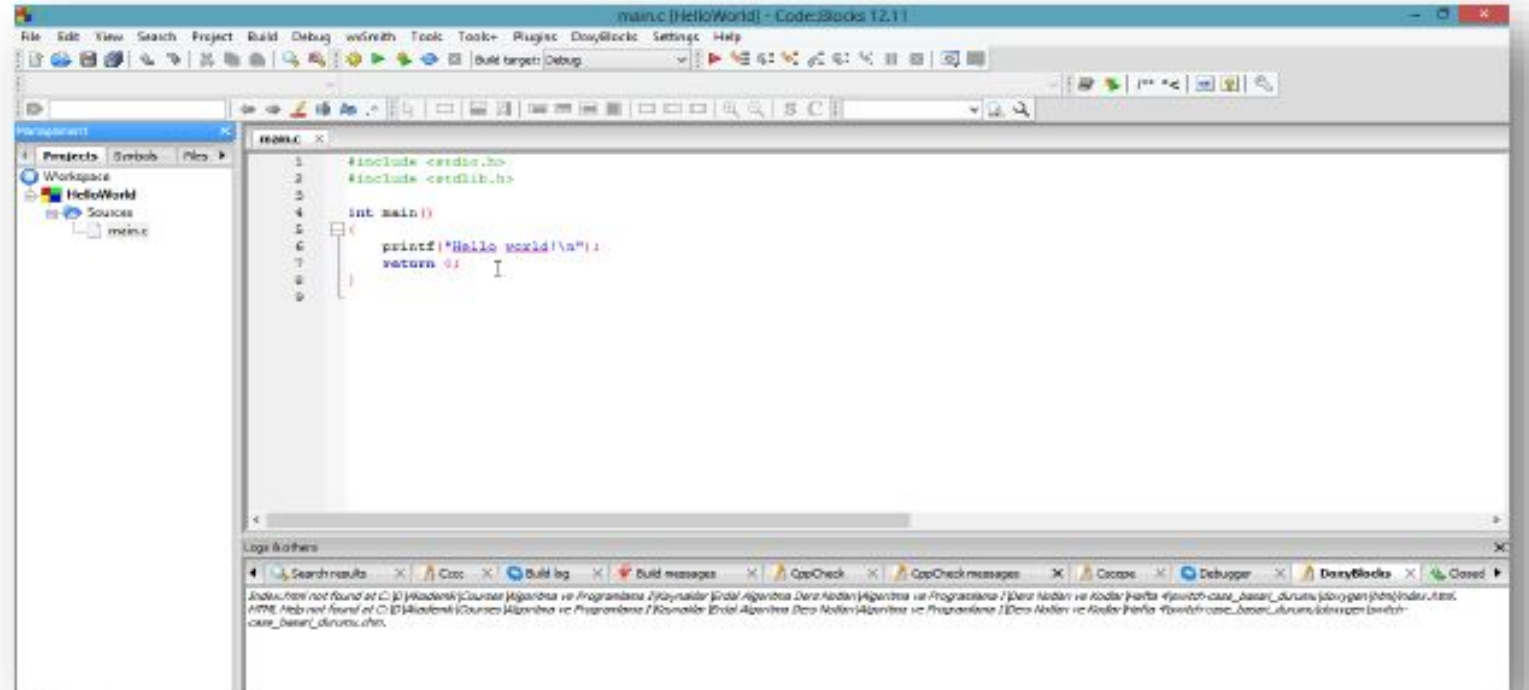
Çalıştırma (execute) Oluşturulan makine dili programının çalıştırılması adımıdır.



Yazılımların Çalıştırılması...

IDE (Integrated Development Environment – Tümleşik Geliştirme Ortamı)

yazılımcının hızlı ve kolay bir şekilde program geliştirebilmesini amaçlayan, geliştirme sürecini organize edebilen ve sürecin verimli kullanılmasına katkıda bulunan araçların tamamını içerisinde barındıran bir yazılım türüdür.

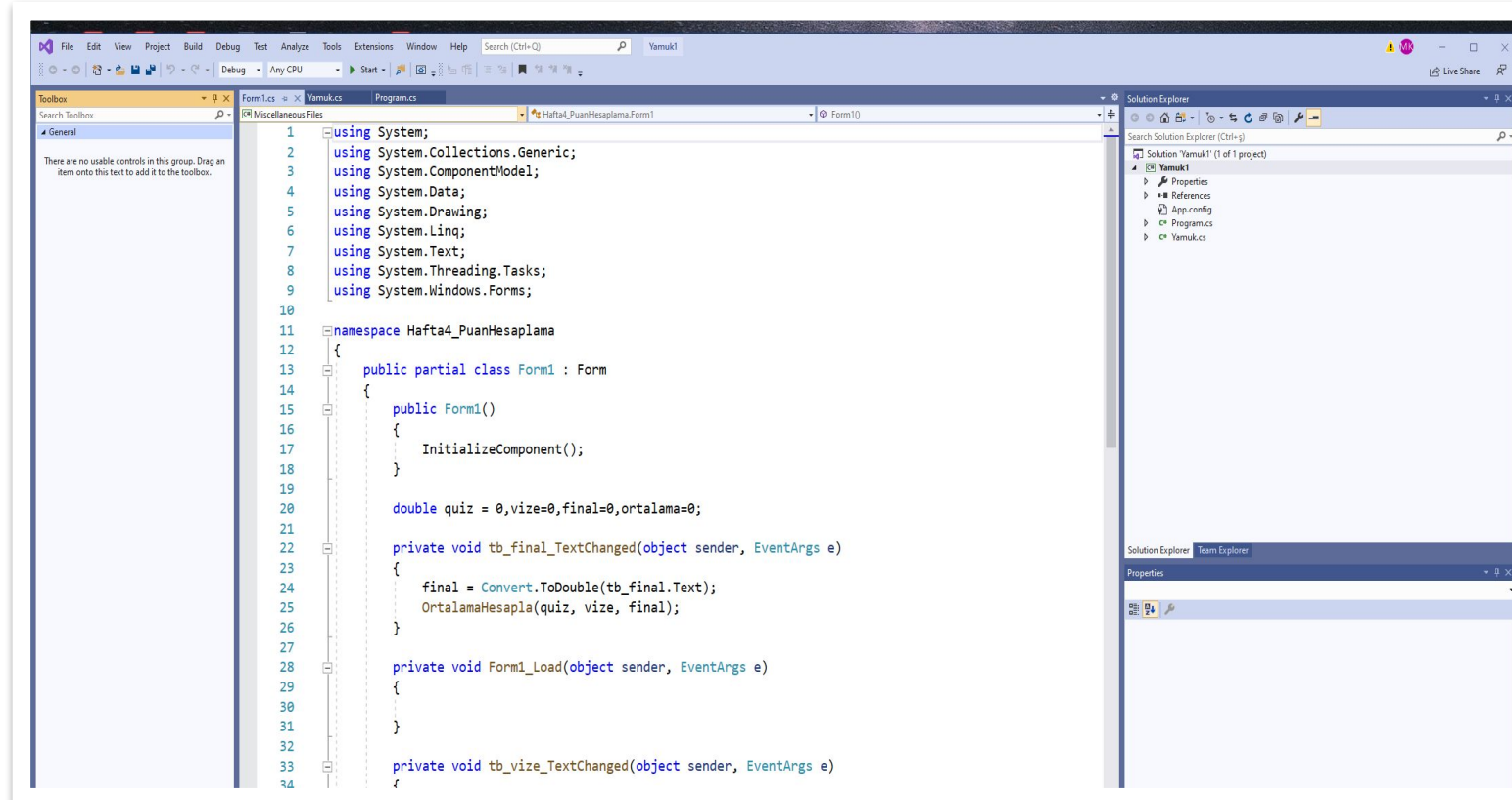


Yazılımların Çalıştırılması...

IDE

Tümleşik geliştirme ortamlarında olması gerekli en temel özellikler aşağıdaki gibidir:

- Programlama diline göre sözdizim renklendirmesi yapabilen kod yazım editörü
- Kod dosyalarını hiyerarşik olarak görülebilmesi
- Tümleşik bir derleyici, yorumlayıcı ve hata ayıklayıcı
- Yazılımın derlenmesi, bağlanması, çalışmaya tümüyle hazır hale gelmesi
- ve daha birçok ek işi otomatik gerçekleştirebilen inşa araçları



Yazılımların Çalıştırılması...

IDE

Bazı tümleşik geliştirme ortamları

- Eclipse, MS Visual Studio, Code::Blocks, Dev-C++, Kdevelop, Netbeans



Yazılımların Çalıştırılması...

Yazılım hataları

- Yazılım geliştirme süreci aşamalarında (analiz, tasarım, kodlama, test, bakım) karşılaşılan hatalardır.
- Doğru hata yönetimi yaparak hata sayıları azaltılabilir ve hata oluştuğunda hızlı bir şekilde müdahale edilebilir.
- Uygulama geliştirme aşamasında hatalar 3 grupta değerlendirilir:
 - I. syntax error - sözdizimi hataları
 - II. run-time error – çalışma zamanı hataları
 - III. logic error (bug) – mantıksal hatalar



Yazılımların Çalıştırılması...

1. syntax error (sözdizimi hataları)

Yazılan programda programlama dili kurallarına aykırı bir takım ifadelerden dolayı karşılaşılabilecek hatalardır.

Düzeltilmesi basit hatalardır.

Hatanın bulunduğu satır derleyici tarafından rapor edilir.

Günümüz IDE'lerinde bu sıkıntılar neredeyse yok denecek kadar azdır. Özellikle kod editörlerinin gelişmiş yazım denetimi sayesinde yazılımcılar söz dizim hatalarını derleme aşaması öncesi bile fark edebilirler.

Eğer bir derlemede syntax error alındı ise obje kod üretilememiş demektir.



Yazılımların Çalıştırılması...

2. run-time error (çalışma zamanı hataları)

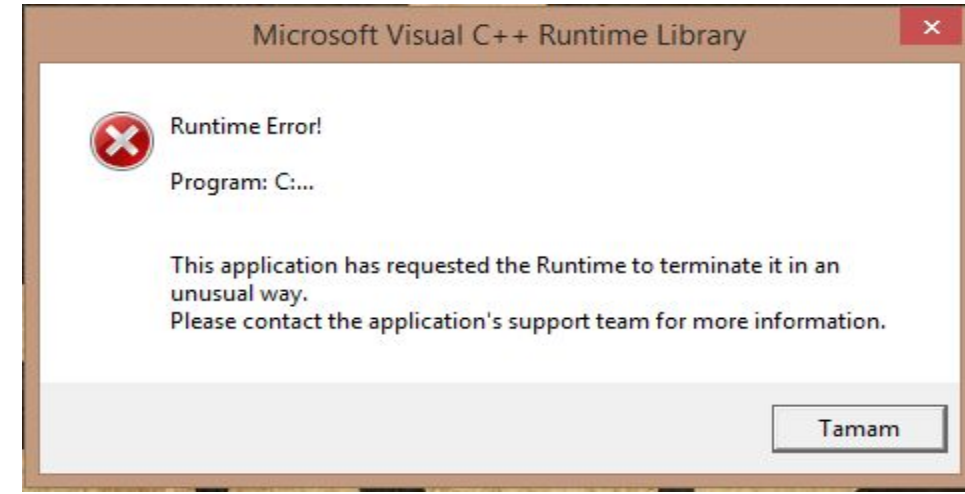
Programın çalıştırılması sırasında karşılaşılan hatalardır.

Ele alınmayan bir takım aykırı durumlar ortaya çıktığında programın işletim sistemi tarafından kesilmesi ile ortaya çıkar. Oluşan hata çoğunlukla çalışan işletim sisteminin dili ile verilir.

Bu tip hataları kullanıcı ele almışsa, program programcının vereceği mesajlarla ve uygun şekilde sonlandırılabilir.

Nerede ve hangi şartlarda ortaya çıkabileceğini bazen kestirmek zor olabilir.

Örneğin **olmayan bir dosya açmaya çalışmak, var olan bir dosyanın üzerine yazmaya çalışmak, olmayan bir bellek kaynağından bellek ayırtmaya çalışmak, olmayan bir donanıma erişmeye çalışmak vb.**



Yazılımların Çalıştırılması...

3. Logic error (Bug) – (mantıksal hatalar)

Karşılaşılabilecek **en tehlikeli hatalardır**. Programlama mantığında bir takım şeylerin yanlış düşünülmesinden kaynaklanır. Hata **test aşamasında veya müşteri kullanımı** sırasında ortaya çıkar. Örneğin:- hesaplanması gereken veya bulunması gereken değerlerin eksik veya yanlış hesaplanması mantıksal bir hatadır. Bu sorunun giderilebilmesi için analiz aşamasına kadar geri dönülmesi gerekebilir. Bazen bu hatanın **nereden kaynaklandığını bulmak çok zor olabilir**.

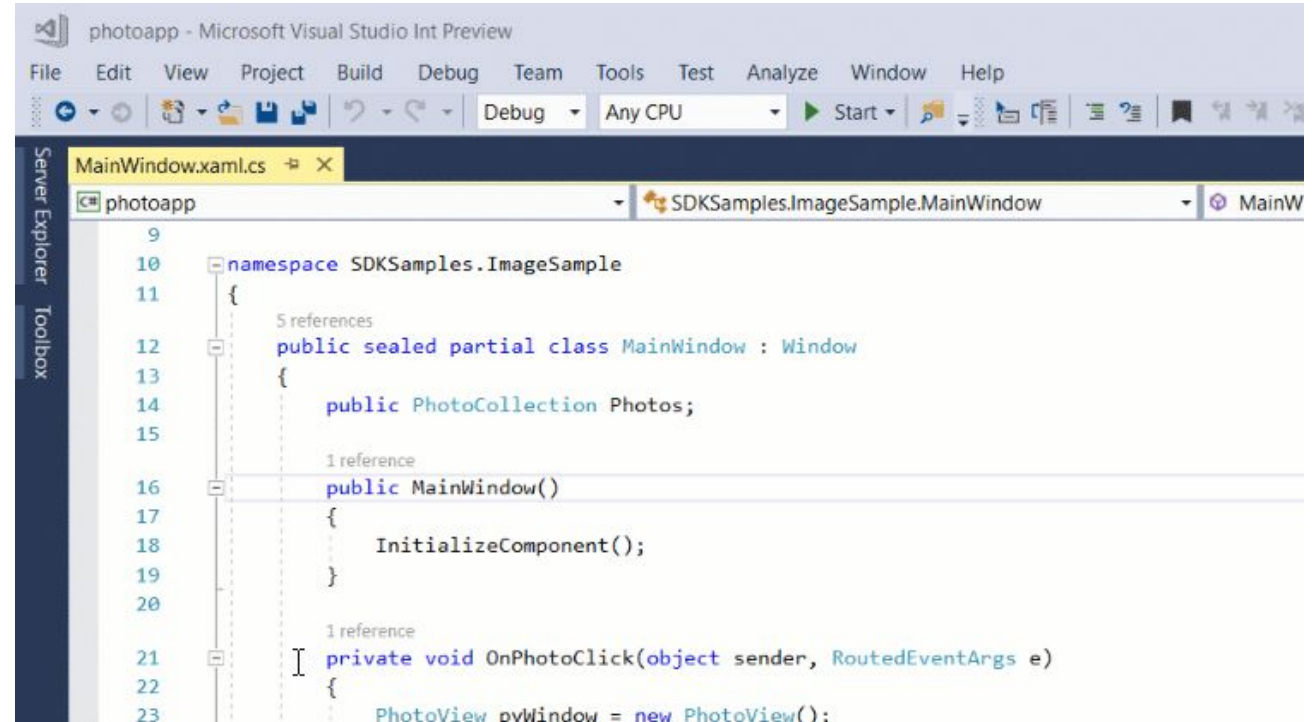
günümüzde en etkin firmalar bile yazılımlarında **bug olduğunu kabul eder** ve zaman zaman bu bugları giderebilmek için ya **yama yazılımları (update, patch)** ya da **yeni bir versiyon** piyasaya sürerler.



Yazılımların Çalıştırılması...

Debug (bugdan arındırma)

- Mantıksal hataları giderebilmek ve yazılımdaki hataları bulabilmek için yapılan işlemin adıdır. Genellikle yazılan programın adım adım ve denetim altında çalıştırılmasıdır.
- İlgili adımlarda değişkenlerin hangi değere sahip olduğunu görmemizi sağlayarak anormal durumları tespit edip bulmamızı sağlar. .



```
photoapp - Microsoft Visual Studio Int Preview
File Edit View Project Build Debug Team Tools Test Analyze Window Help
Debug Any CPU Start
Server Explorer Toolbox
photoapp SDKSamples.ImageSample.MainWindow MainW
9
10 namespace SDKSamples.ImageSample
11 {
12     5 references
13     public sealed partial class MainWindow : Window
14     {
15         public PhotoCollection Photos;
16
17         1 reference
18         public MainWindow()
19         {
20             InitializeComponent();
21
22         1 reference
23         private void OnPhotoClick(object sender, RoutedEventArgs e)
24         {
25             PhotoView pvWindow = new PhotoView();
```

Programlama Teknikleri

□ Yapısal / modüler Programlama tekniği

- Structured Programming

□ Nesne Yönelimli Programlama tekniği

- Object Oriented Programming

Yapısal/Modüler Programlama

- Programcı doğrudan probleme odaklanır ve problem çözümüne ilişkin fonksiyonları / yöntemleri geliştirir.
- Ana fonksiyon/yöntem (main) programı başlatır ve her biri özel görevleri yerine getiren yöntemler çağrılarak yapılması gerek işlem gerçekleştirilir
- Yöntemlerin ihtiyaç duyduğu veriler genellikle veri tabanlarında ya da (her taraftan erişilebilen) değişkenlerde saklanır.
- Öğrenci, açılan dersler, not girişleri, vb. işlemleri gerçekleştiren bir bilgi sistemi düşünelim.

Alt yordamlar

ogrenciEkle()

notGoruntule()

notGir()

ogrenciSil()

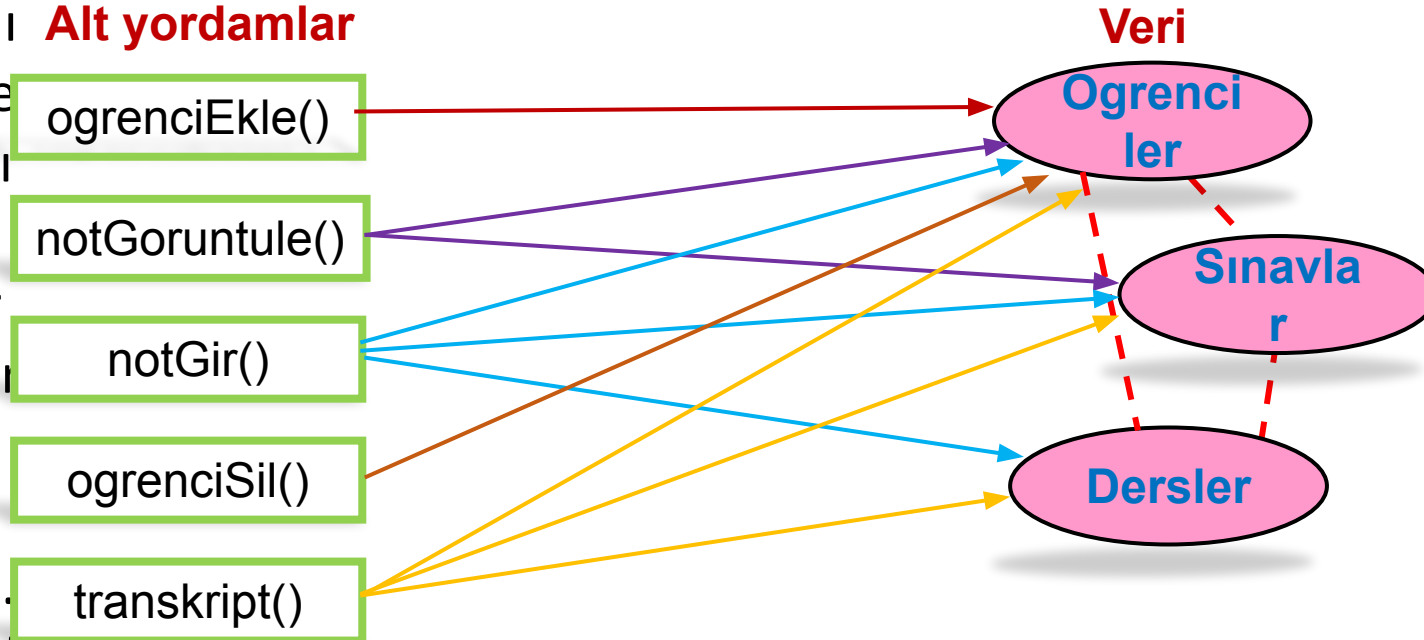
transkript()

Veri

Oğrenciler

Sınavlar

Dersler



Yapısal/Modüler Programlama

- ❑ Sistem büyüdükçe ilişkiler ve bağımlılık daha da karmaşıklaşır
- ❑ Hata bulma zorlaşır
- ❑ Program içerisinde değiştirme, ekleme, çıkarma, vs. yapmak zorlaşır ve beklenmeyen etkilere neden olabilir
 - Öğrenciler tablosunda öğrencinin doğum tarihi iki haneli olsun
 - Bu alanı dört haneli yapmak istediğimizi varsayalım
 - Öğrenciler tablosu sınavlar ve dersler tabloları ile ilişkili olduğundan beklenmeyen bir sorun çıkabilir
 - Tüm yöntemler öğrenciler tablosunu bir şekilde kullanıyor
 - O nedenle ogrenciEkle() yöntemi sorun çıkaracaktır

Alt yordamlar

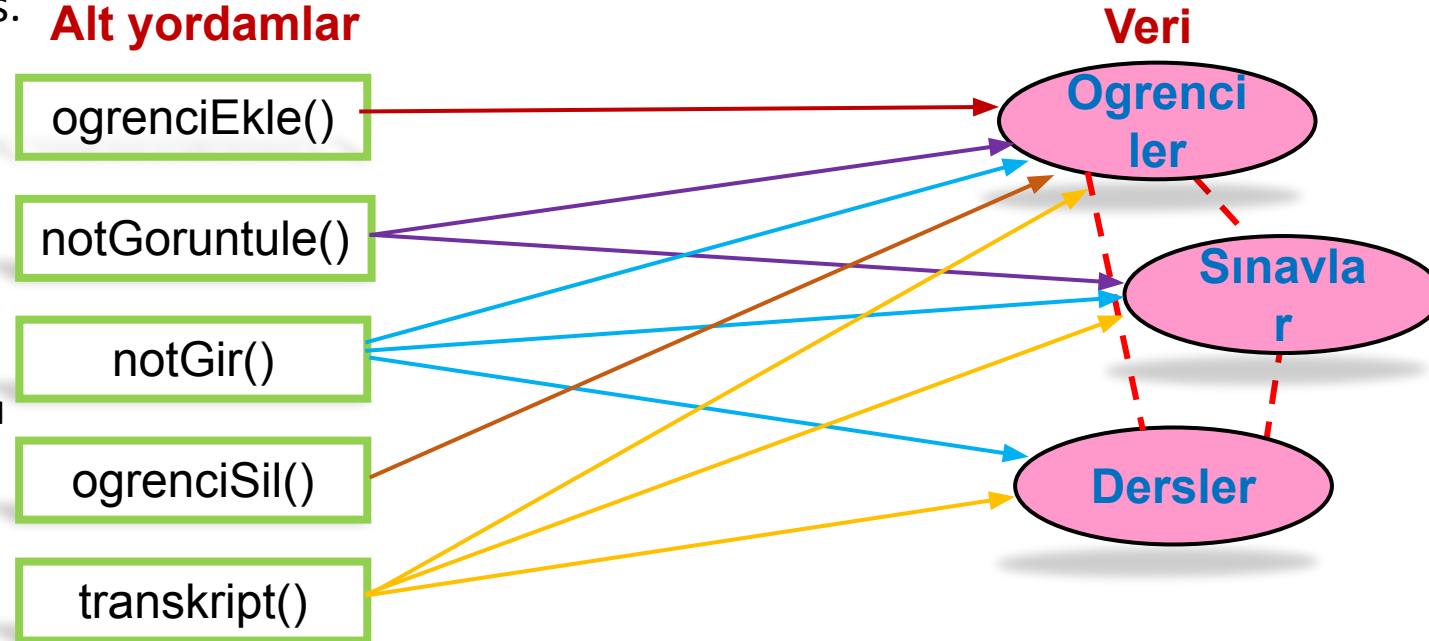


Veri

Oğrenciler

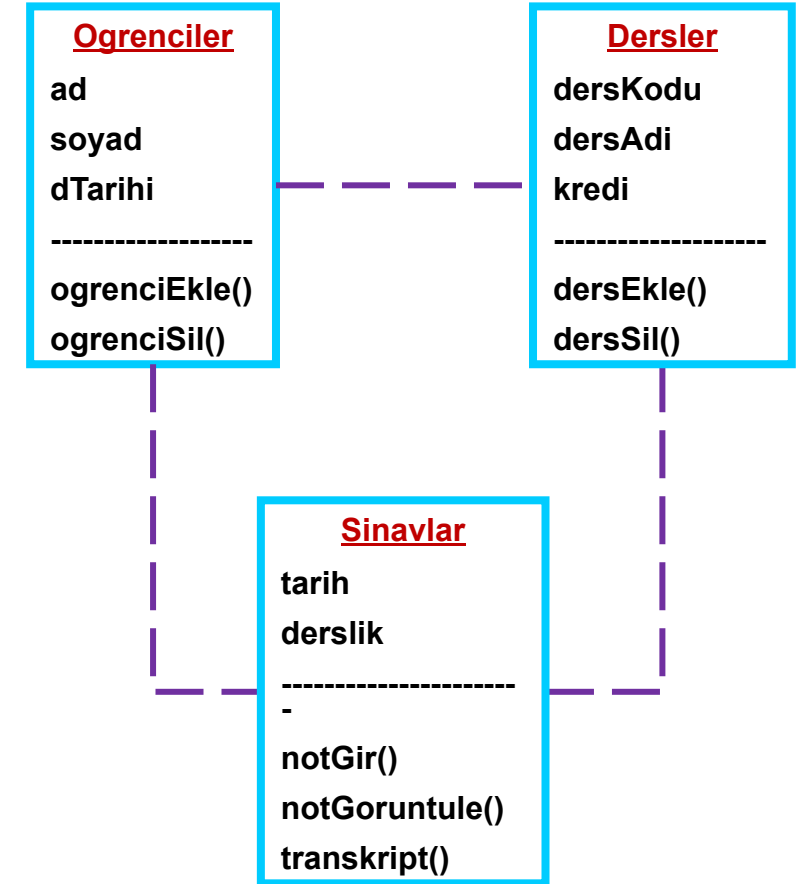
Sınavlar

Dersler



Nesneye Dayalı Programlama

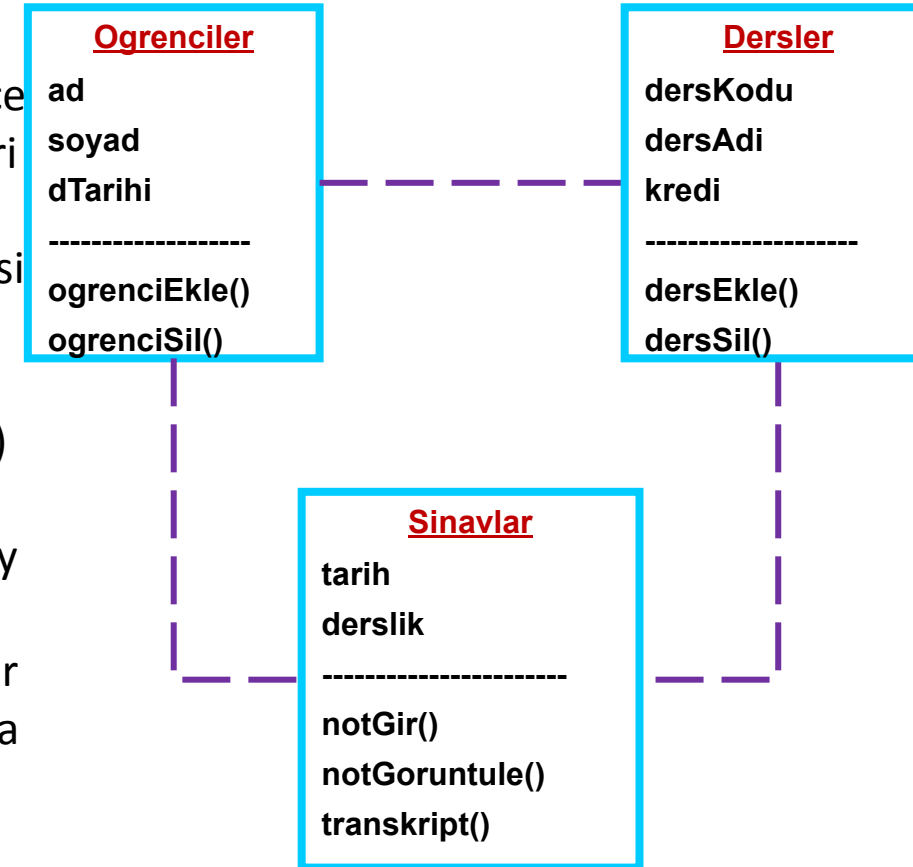
- Yapısal teknikte programcı doğrudan probleme odaklanır ve problemin çözümüne ilişkin yöntemleri geliştirir. Nesneye yönelimli programlama tekniğinde ise temel bileşen nesnedir ve programlar nesnelerin birlikte çalışmasından meydana gelir. Nesne hem veriyi hem de bu veriyi işleyen fonksiyonları içerir. Programcılar dikkatlerini nesneleri oluşturan sınıfları geliştirmeye yoğunlaştırır.
- Yapısal teknikte bir fonksiyon herhangi bir görevi yerine getirmek için veriye ihtiyaç duyarsa, gerekli veri parametre olarak gönderilir. NDP de ise yerine getirilmesi gereken görev nesne tarafından icra edilir ve fonksiyonlar verilere parametre gönderimi yapılmaksızın erişebilirler.



Nesneye Dayalı Programlama

Özellikleri / Avantajları

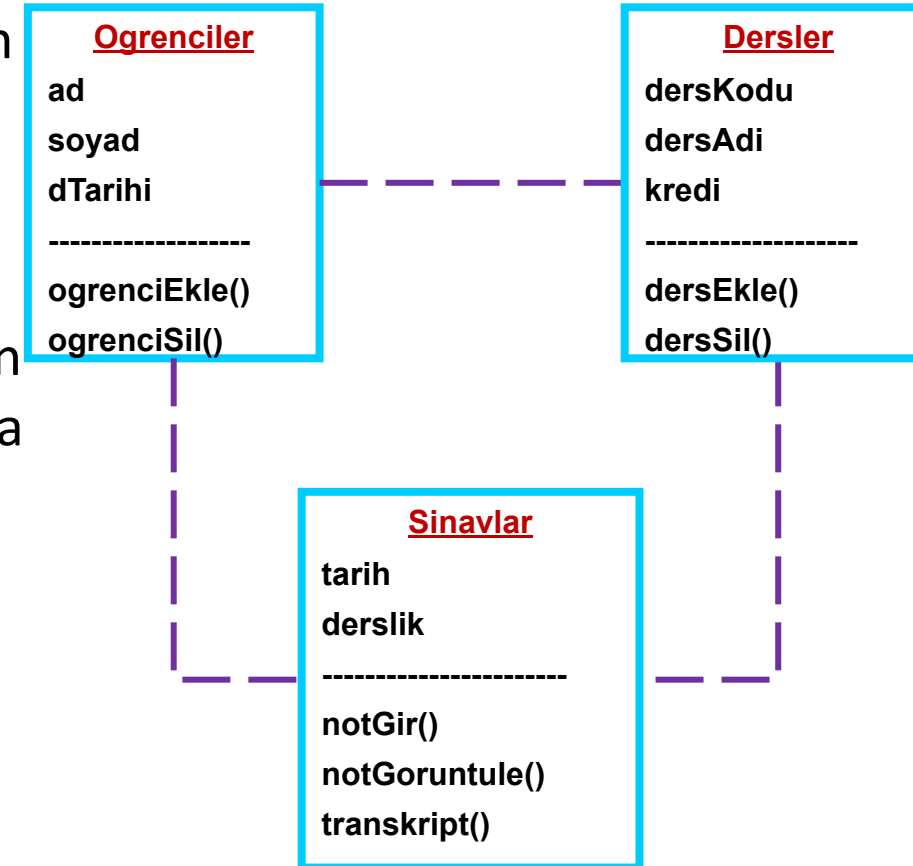
- ❑ Problemler daha kolay tanımlanıp çözülebilir. Gerçek dünya düşünülerek geliştirilmiştir. Geliştirme süreci daha kolay olur.
- ❑ Bilgi Gizleme (Information Hiding, Data abstraction)
 - Nesne içerisindeki işlemler (nasıl) diğer nesnelerden soyutlanır-sadece ne yapacağını bilirler. Nesne içerisindeki değişiklik diğer nesneleri etkilemez. Dolayısıyla bakım aşaması daha kolay olur.
 - Gereksiz ayrıntılarla uğraşılmaz, probleme odaklanılır (arabanın gitmesi için gaza basmak yeterli). Daha hızlı geliştirme süreci sağlar.
- ❑ Modüler Programlama (Modular Programming)
 - Nesneler birbirinden tamamen bağımsız (veri + fonksiyon) (encapsulation, responsibility driven design)
 - Büyük ve karmaşık bir problem küçük parçalara ayrılarak daha kolay çözülebilir. Geliştirme ve bakım daha kolay olur.
 - Programların geliştirilmesi daha hızlı, geliştirilen bir nesne diğer programlarda da rahatlıkla kullanılabilir. Hata bulma-bakım daha kolaydır (Bisikleti başkasına verdiğimiz zaman da çalışır).
 - Geliştirme sürecinde grup çalışmalarına olanak sağlanır.



Nesneye Dayalı Programlama

Özellikleri / Avantajları

- ❑ Kodların Tekrar Kullanımı (Code Reuse)
 - Nesneler başka programlara kolaylıkla aktarılabilir. Bakım ve geliştirme zamanı/maliyeti düşer
 - Kalıtım, Çok biçimlilik
- ❑ Hata Bulma – Bakım/Onarım (Maintenance)
 - Bileşenler arasındaki ilişkiler açık olduğundan (veri+fonksiyon aynı yapı içerisinde) güncelleme, hata bulma ve bakım daha kolay
- ❑ Tasarım Desenler (Design Patterns)
- ❑ Günümüzdeki en iyi yaklaşım. Gelecekte ?



Programlama Dillerinin Tarihçesi



1940'lı yıllarda ilk defa elektrik ile çalışan bilgisayarlar;

Çok basit işlemler yapabildiklerinden programlanmaları gerekmiyordu.

Hafızası oldukça limitli.

Genellikle birden fazla teknisyenin hızlıca kabloların yerlerini değiştirmesi gerekiyordu.

İşlemler arasında geçiş yapma çok uzun sürebilmekte.

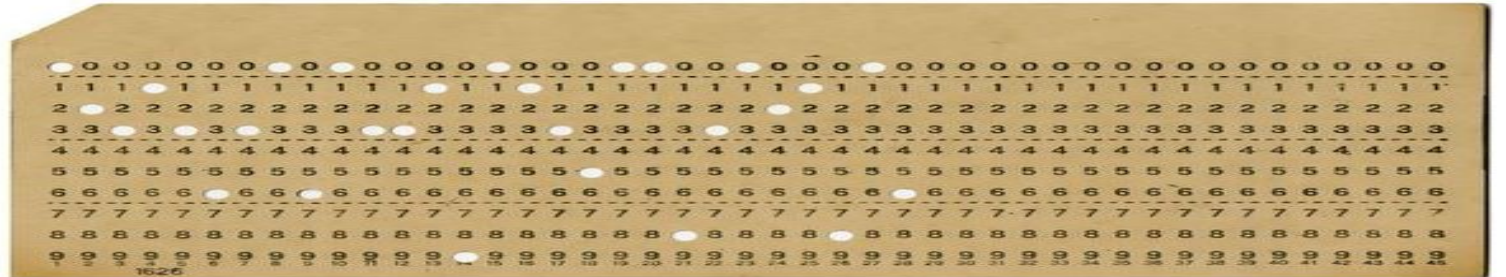
Delikli kart sistemine geçildi.

Elektriği iletmeyen bir materyalden yapılıyordu

Belirli yerlerde delikler vardı ve kablo bağlantılarının arasına yerleştiriliyorlardı.

Dokuma tezgahları, otomatik piyanolar ve daha sonraları bazı matbaalar, delikli kart teknolojisini kullanmışlardı.

<https://www.youtube.com/watch?v=fnb7EqfykF4>



Delikli kartların görünüşü – Resim: computerhistory.org

Programlama Dillerinin Tarihçesi



Bilgisayar dünyasında, bu kartlar bilgisayarları binary, yani ikili sistemde kodlamak için kullanılıyorlardı. 0 (sıfır) ve 1 (bir) olmak üzere iki duruma göre kodlanabilmekteydi.

0: akım geçmeyen, 1: akım geçen.

İkili sistemin bir diğer adı da **makine dilidir**.

Günümüzde bilgisayarda yapılan bütün işlemler en sonunda “0” ve “1” lere dönüştürülerek gerçekleştirilir. (Kuantum Bilgisayarları hariç) Bilgisayardaki mikroişlemciler; açma ve kapama anahtarı gibi çalışan, içinde milyonlarca transistörün bulunduğu elektronik devrelerdir.

Bilgisayardaki merkezi işlemci, işlemleri en basit sistem olan ikili (binary) düzen yani sadece “0” ve “1” sayılarını kullanarak yapar. Sayı grupları üzerinde işlem yapmak için, işlemci üzerinde bir takım komut listeleri yani programlar bulunur. Bilgisayarlar bu programlar sayesinde işlemlerini gerçekleştirirler.

```
0100100001100101011011000110110001101111001000000101011101101111011100100110110001100100
```

İkili sistemde “Hello World” metni

Programlama Dillerinin Tarihçesi



Makina kodlarını kullanarak bilgisayara program yüklemek çok zor bir iştir.

Kart okuyucu'ların ortaya çıkışı bu işlemi oldukça kolaylaştırdı.

Kart okuyucudan gelen sinyali algılayıp makina koduna çeviren programlar günümüz işletim sistemlerinin atası sayılabilir.

Assembly gelişimi.

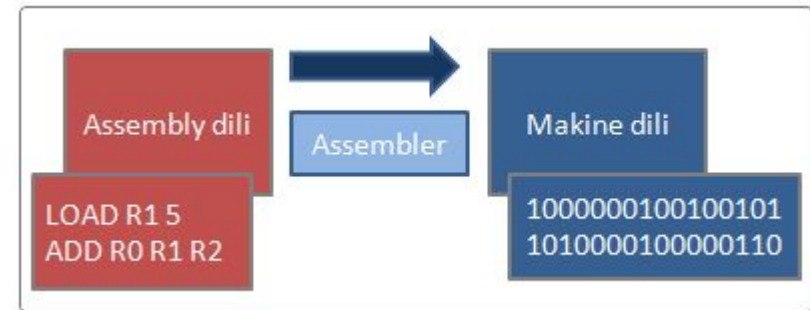
Program yazmayı oldukça kolaylaştırdı.

En önemli özelliği artık yazılan programlar okunabilir programlar oldu.

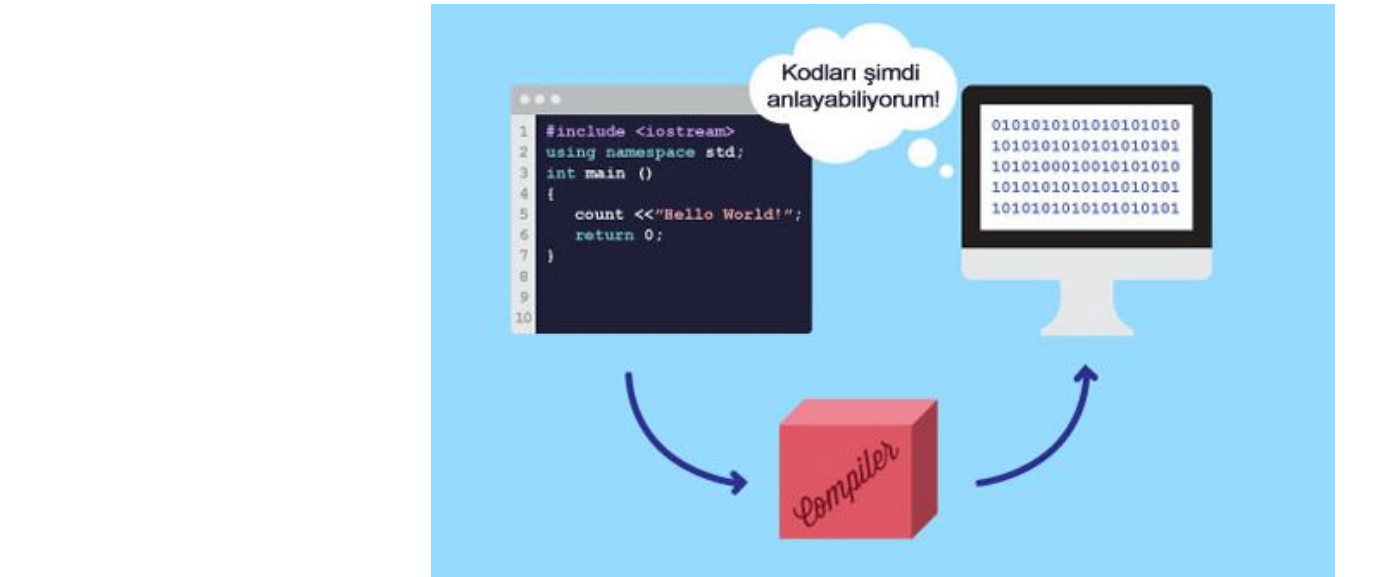
Makine bağımlı.

```
org 0x100
mov dx, msg
mov ah, 9
int 0x21
mov ah, 0x4c
int 0x21
msg db 'Hello, World!', 0x0d, 0x0a, '$'
```

Montaj dilinde "Hello World" metnini ekrana yazdıran kod



- ne benzer tanımları olan programlama dili.
- yönetimi, ve benzeri diğer karmaşık görevleri kullanıcıya
- ağı için bu programlar öncelikle bilgisayarların anlayacağı



Programlama Dillerinin Tarihçesi



İnsan Diline Yakın Programlama Dillerinin Doğuşu

FORTRAN (1957); IBM evrensel bir çözüm bulmak ve herkesin her makina için program yazabilmesine olanak sağlayacak bir dil geliştirdi: FORTRAN (FORMula TRANslation).

- Özellikle matematiksel hesaplamalar için geliştirilmiştir.
- FORTRAN çok kolay kullanılır ve kolay okunur bir dil oldu.
- FORTRAN programlama dili, önceden yazılmış programların daha yeni bilgisayarlarda da çalıştırılabilmesini olağan kıldığından hızla yaygınlaştı.

```
Program Hello  
Print *, "Hello World!"  
End Program Hello
```

Modernleştirilmiş FORTRAN dilinde "Hello World" metnini ekrana yazdıran kod

Programlama Dillerinin Tarihçesi



İnsan Diline Yakın Programlama Dillerinin Doğuşu

LISP (1958) ; yılında recursive (kendisini tekrarlayan) fonksiyonlar düşünülerek geliştiriliyordu.

- ❑ FORTRAN'da bir kod, bir seri şeklinde yazılmak zorundayken; LISP'te yazılan bir kod ayrı fonksiyonlar ve döngüler şeklinde yazılabiliyordu.
- ❑ Birden fazla işlem gerektiren durumlarda parantez kullanarak bu işlemleri birleştirebiliyordu.
- ❑ LISP, derlenerek çalışmanın yanı sıra, yorumlanarak (Interpreted), çalışma sırasında makine diline çevirilerek de çalıştırılabiliyordu.
- ❑ Fonksiyonları yineleme gibi özellikleri sayesinde, veri işleme, ya da yapay zeka geliştirme çalışmaları gibi ağır ve tekrarlayıcı görevlerde kullanılan bir dil oldu.
- ❑ Günümüzde, C-LISP (Common LISP) ve Scheme olarak adlandırılan, orijinal LISP dilinin geliştirilmiş versiyonları mevcuttur, hatta LISP dili hala yapay zeka araştırmalarında kullanılmaktadır.

```
; LISP  
BEGIN DEFUN (hello ())  
  (PRINT (LIST 'HELLO 'WORLD))  
END(hello)
```

Modern C-LISP dilinde "HELLO WORLD" metnini ekrana yazdıran kod

Programlama Dillerinin Tarihçesi



İnsan Diline Yakın Programlama Dillerinin Doğuşu

ALGOL (ALGOritmic Language- 1958-60) ; FORTRAN'da karşılaşılan bazı problemlerin çözülmesi ve karmaşık kodların daha anlaşılabilir olması amaçlanmıştı.

- ALGOL dili ile ilk defa kod blokları denen kavramı ortaya attı, ve kendisinden önce gelen dillerdeki “BEGIN” ve “END” tanımları kaldırıldı.
- iç içe fonksiyonlar.

COBOL (COmmon Business Oriented Language - 1959); Programların daha kolay yazılabilmesi ve İngilizce'ye mümkün olduğunca benzer bir programlama dili amaçlanıyordu.

- Kendi içerisinde pek çok fonksiyonu barındırıyordu.
- İngilizce'ye olan benzerliği ile hem iş yönetimi, hem de elektronik parça üretimi sektörlerince uzun yıllarca kullanılmıştı.
- Günümüzde hala kullanılan ve güncellenmeye devam eden bir dil.

```
OUTPUT(4,('Hello World'))
```

ALGOL dilinde "Hello World" metnini ekrana yazdıran kod

```
Identification Division.  
Program-ID. HelloCOBOL.  
Data Division.  
Procedure Division.  
Main-Paragraph.  
Display "Hello World!"  
Stop Run.
```

COBOL dilinde "Hello World" metnini ekrana yazdıran kod

Programlama Dillerinin Tarihçesi



İnsan Diline Yakın Programlama Dillerinin Doğuşu

BASIC (Beginners All Purpose Symbolic Instruction Code - 1964) ;

- Microsoft.
- İster yorumlanarak, ister derlenerek çalıştırılabiliyordu.
- İsmine bakarak anlaşılabilirdiği gibi, BASIC, basit bir dildi ve bilgisayarlar ile ilgili teknik bilgisi çok iyi olmayan kitleye de hitap ediyordu.
- Kısa bir süre sonra programlamayı yeni öğrenenler arasında popülerleşen bir dil oldu.

```
10 PRINT "HELLO WORLD"
```

BASIC dilinde "HELLO WORLD" metnini ekrana yazdıran kod

PASCAL (1970)

- BASIC gibi, programlamayı yeni öğrenenler için kolay bir dil.
- Hem bir derleyiciyle hem de bir yorumlayıcı aracılığıyla çalıştırılabiliyordu. Daha anlaşılır ve hatalara karşı daha korumalı programlar yazmasını hedeflemekteydi.
- PASCAL, kolay öğrenilmesi sayesinde bazı ufak işletmeler tarafından tercih edilmişti.

```
Program Hello;  
Begin  
  Write('Hello World');  
  Readln;  
End.
```

PASCAL dilinde "Hello World" metnini ekrana yazdıran kod

Programlama Dillerinin Tarihçesi



İnsan Diline Yakın Programlama Dillerinin Doğuşu

C (1972) ;

- UNIX işletim sistemleri için tasarlanan bir programlama diliydi
- UNIX, ilk defa birden fazla görevi aynı anda yürütebilen bir işletim sistemi olacak bir tasarıydı.
- Aynı anda birden fazla veriyi işleyebilmek, ve verileri karıştırmadan depolayabilmek için gelişmiş bir veri adres pointer (işaretçi) sistemi bulunan C dili bu şekilde geliştirilmeye başladı.
- C dilinin gelişmiş hafıza yönetimi özellikleri, kendisinin hızlıca diğer sektörlerde de yayılmasını sağladı.
- C dili o kadar popüler bir dil oldu ki, kendisinden sonra gelen hemen hemen her cihaz C dilini destekler hale geldi.
- Günümüzde, C programlama dilinin hafıza yönetimini biraz daha kolay hale getiren C++ (1983), ve C dilinin obje odaklı programlama için optimize edilmiş versiyonu olan OBJECTIVE-C (1983) gibi pek çok versiyonu mevcuttur, ve C dili hala pek çok sektör tarafından pek çok alanda kullanılmaktadır.
- C++ dili, C dilinin açıklarını kapatarak, kendisini, uzun bir süre boyunca en popüler yazılım dili haline getirmişti. Şu anda bile C++ dili en çok tercih edilen dillerden bir tanesidir.

```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    printf("Hello World\n");
    return EXIT_SUCCESS;
}
```

C dilinde "Hello World" metnini ekrana yazdıran kod

C++

```
#include <iostream>
int main()
{
    std::cout << "Hello World";
    return 0;
}
```

C#

```
class Hello {
    static void Main() {
        System.Console.WriteLine("Hello World");
    }
}
```

C++ ve C# dillerinde "Hello World" metnini ekrana yazdıran kod

Programlama Dillerinin Tarihçesi



İnsan Diline Yakın Programlama Dillerinin Doğuşu

Python (1991),

Yorumlanarak çalıştırılan bir programlama dili.

Anlaşılabilir ve akıcı bir şekilde okunabilir kodlar olması amaçlandı.

Basit yapısı.

en büyük özelliklerinden birisi dinamik hata denetimi özelliğinin bulunması. Python'un yorumlanarak çalıştırılan bir dil olması sayesinde, Python dilinde yazılan programlarda bir hata olsa bile, bu programların hatasız kısımlarının çalıştırılıp, hata olan kısma kadar programın test edilebilmesini sağlıyordu.

Hafıza yönetimini tamamıyla kendi içerisinde otomatik olarak gerçekleştirilmekte.

Python dili, sıklıkla web tabanlı uygulamalarda kullanılan bir dil olmasına rağmen, esnekliği sayesinde, Python, pek çok farklı alanda, pek çok farklı yöntemle kullanılabilen bir dil haline geldi.

Ders içerisinde programlama dilleri arasındaki farklılıklara değinmek ve çeşitlilik sağlaması amacıyla bazı uygulamalar Python dili kullanılarak da örneklenebilir.

```
print("Hello World")
```

Python dilinde "Hello World" metnini ekrana yazdıran kod

Programlama Dillerinin Tarihçesi



İnsan Diline Yakın Programlama Dillerinin Doğuşu

Java(1995),

- Nesne odaklı.
- Farklı sürümlere sahip ve sürümler arasında ufak tefek sözdizimi farklılıkları mevcut.
- Java kullanılarak yazılan programların hemen hemen her cihaza uyumlu olup, her cihazda çalıştırılabilmesi idi.
- Hatta, Java'da yazılan bir program, Java tarafından desteklenen başka bir cihaz için tekrardan derlenmesi gerekmiyordu.
- Java, bu derece uyumlu olabilmeyi, işletim sistemleri arasında farklılık gösteren komutlardan kaçınarak başarabildi.
- Java, ayrıca arkaplan görevleri ve grafiksel arayüzlerin oluşturulması gibi farklı alanlarda da kullanılan bir dildi.
- Günümüzde Java, en çok kullanılan programlama dillerinden biridir

Java

```
public class Hello {  
    public static void main(String []args) {  
        System.out.println("Hello World");  
    }  
}
```

Programlama Dillerinin Tarihçesi



İnsan Diline Yakın Programlama Dillerinin Doğuşu

C# programlama dili (2000),

- Microsoft,
- Nesne yönelimli
- Adının temeli C ve C++ dilinden gelir. C dilinde bir tamsayı değişkenin değerini artırmak için “++” operatörü kullanılır. Bu yüzden C dilinden sonra geliştirilen ve Nesne Yönelimli (Object Oriented) özellikleri taşıyan yeni dil için, C'nin bir fazlası anlamında C++ adı kullanılmıştır. C++'tan sonra geliştirilen C# dili ise adını; ((C++)++) ifadesinde artı işaretlerinin bir araya gelmesinden alır.
- Dönemin en yaygın ve en beğenilen Nesne Yönelimli dili olan Java'nın söz dizimini temel almıştır.
- C# dilinin tasarımındaki başlıca hedefler şunlardır:
 - Basit, modern, her tür kullanım amacına uygun ve nesne yönelimli bir dil olması,
 - Güçlü tiplendirme kontrolü, izin sınırları kontrolü ve tanımlanmış değişkenlerin kullanım tespiti gibi özellikler barındırması,
 - Programcı portatifliğine sahip olması,
 - Sunucu ve gömülü sistemler için tasarlanmış olması,
 - C, C++ ve Java dilini kullananların rahatlıkla alışabilecekleri şekilde tasarlanmış olması.

```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    printf("Hello World\n");
    return EXIT_SUCCESS;
}
```

C dilinde "Hello World" metnini ekrana yazdıran kod

C++

```
#include <iostream>
int main()
{
    std::cout << "Hello World";
    return 0;
}
```

C#

```
class Hello {
    static void Main() {
        System.Console.WriteLine("Hello World");
    }
}
```

C++ ve C# dillerinde "Hello World" metnini ekrana yazdıran kod

Programlama Dillerinin Tarihçesi



İnsan Diline Yakın Programlama Dillerinin Doğuşu

Swift (2014) ;

Objective c alternatifi olarak Apple tarafından geliştirildi.

Apple tarafından iOS ve OS X platformlarına iOS ve Mac uygulamaları geliştirmek için oluşturulmuş, öğrenilmesi kolay, nesne yönelimli ve güçlü bir dildir.

Objective-C, Rust, Haskell, Ruby, Python gibi birçok programlama dilinden yararlanılarak yazılmış kullanımı kolay ve açık kaynaklı bir dildir.

<https://www.apple.com/tr/swift/>

KOTLIN (2011);

JetBrains'in geliştirdiği bir programlama dilidir.

“Performans ve güvenliği feda etmeden Java’dan daha özlü kodlar yazmayı sağlamak için bu dili geliştirdik”

Nesne yönelimli.

Sunucu ve istemci tabanlı web uygulamaları geliştirmeyi desteklemektedir.

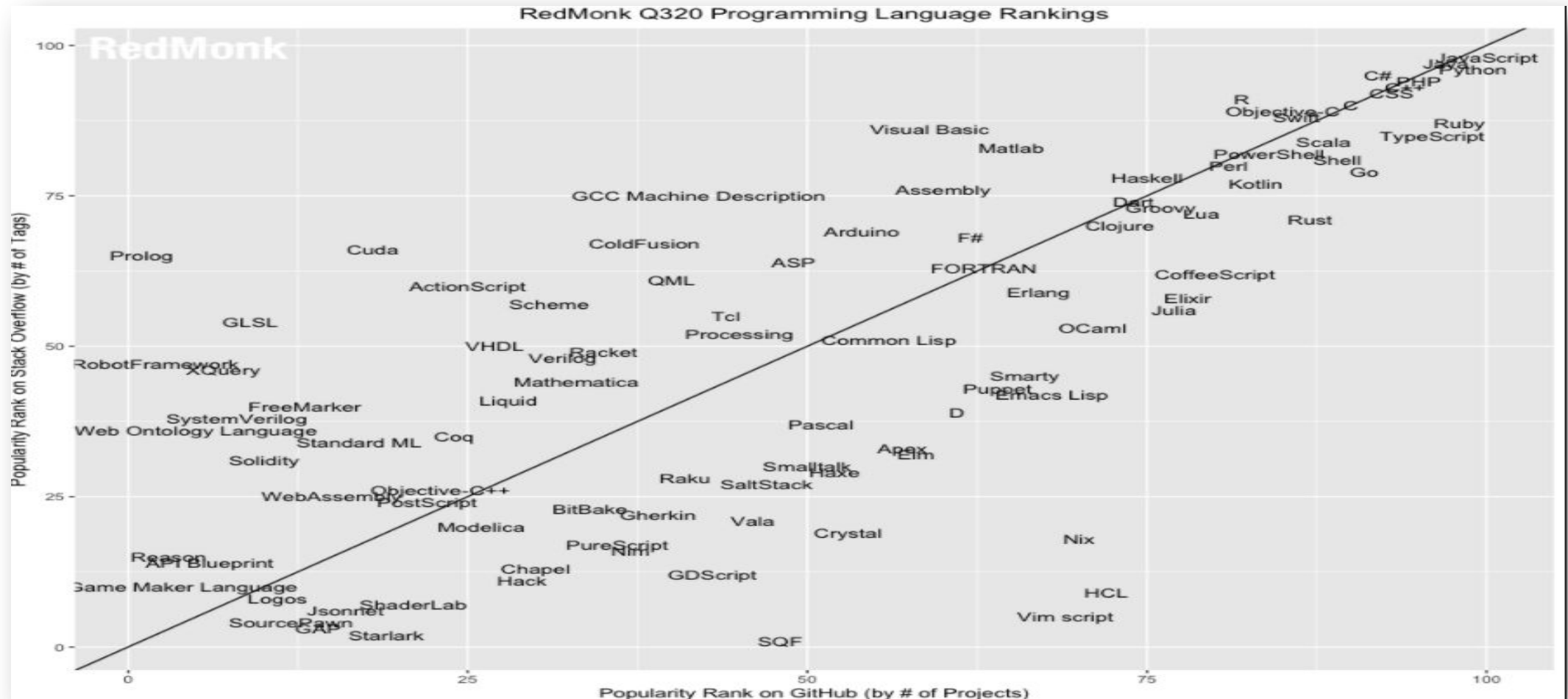
Kotlin dilini öne çıkaran en önemli etken ise : Google Şirketinin Android Developer bölümünün bu dile güvenmeleri ve Android uygulamaları geliştirmek için desteklemeleridir.

Google tarafından Android tarafında destekendiği duyuruldu.

<https://github.com/jetbrains/kotlin>

Yeni nesil diller?

Dart, Go, Solidity , Elm....



Referanslar

C.Öz, C.Bayılmış, G.Çit «Programlamaya giriş» ders notları

N.Taşbaşı ders notları

<https://blog.metu.edu.tr/e224336/2019/03/10/kisaca-programlama-dillerinin-tarihi-ye-niden-paylasim/>

<http://www.baskent.edu.tr/~tkaracay/etudio/agora/bt/pe.html>

Deniz Kılınç Ders Notları

<https://ourcodingclub.github.io/tutorials/fortran-intro/>

<https://redmonk.com/sogady/2020/07/27/language-rankings-6-20/>

<http://bilgioloji.com/pages/yazilim/kod/program/giris/assembly-dili-ne-demektir/>

<https://wmaraci.com/nedir/compiler>

<https://docs.microsoft.com/tr-tr/visualstudio/debugger/debugger-feature-tour?view=s-2019>

<https://www.netowsolutions.com/beware-of-software-bugs/>

<https://www.softwaretestinghelp.com/types-of-software-errors/>

